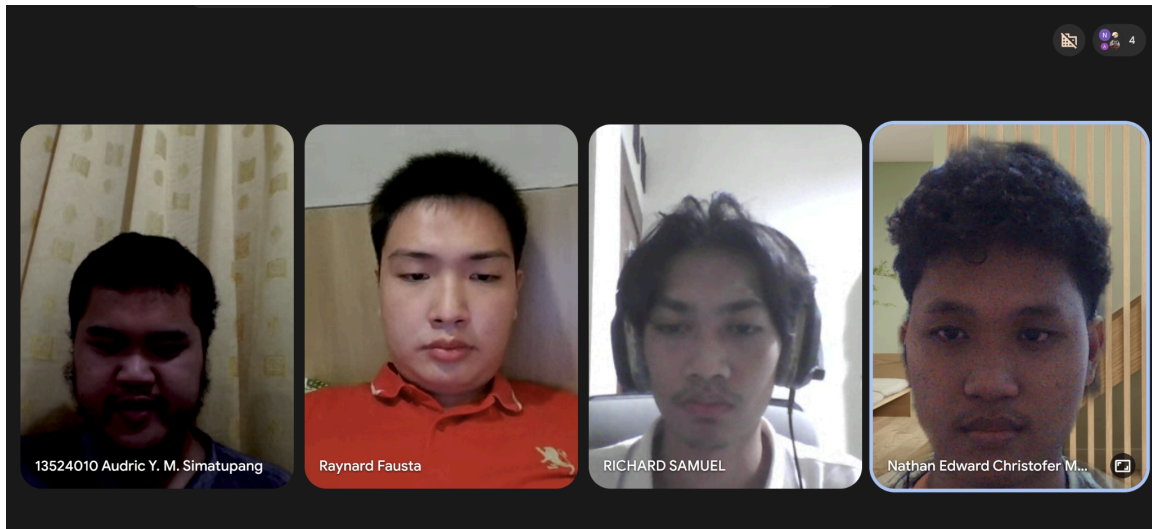


LAPORAN TUGAS BESAR
MATA KULIAH TEORI BAHASA FORMAL DAN OTOMATA
(IF2224)



TUGAS BESAR MILESTONE 3 : SEMANTIC ANALYSIS
KELOMPOK SHT-K02

Anggota :

13524010 - Audric Yusuf Maynard Simatupang
13524052 - Raynard Fausta
13524062 - Nathan Edward Christoper Marpaung
13524112 - Richard Samuel Simanullang

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2026

DAFTAR ISI

LAPORAN TUGAS BESAR.....	1
DAFTAR ISI.....	2
Teori Singkat.....	0
Semantic Analysis.....	0
Abstract Syntax Tree (AST).....	0
Perancangan & Implementasi.....	2
Perancangan.....	2
Implementasi.....	3
AST_Builder.....	3
Struktur Data Symbol_Table.....	3
AST_Decorator.....	3
Pengujian.....	3
Testing 1.....	3
Kesimpulan dan Saran.....	4
Kesimpulan.....	4
Saran.....	4
Lampiran.....	5
Link Repositori.....	5
Pembagian Tugas.....	5
Referensi.....	6

Teori Singkat

Semantic Analysis

Analisis semantik adalah tahapan terakhir pada front-end compilation(interface antara kode sumber dan transformasinya sehingga dapat dijalankan). Tahapan ini bertujuan untuk memastikan deklarasi dan pernyataan dalam suatu program telah benar secara semantik, yaitu maknanya jelas dan konsisten terhadap aturan, serta tipe data yang digunakan memang berlaku dalam bahasa pemrograman tersebut.

Analisis semantik terdiri atas:

- **Type Checking**

Proses untuk memastikan setiap tipe data yang digunakan valid. Hal ini bertujuan agar nilai atau variabel yang digunakan didefinisikan terhadap tipe data yang kompatibel dan hanya boleh dikenai operasi yang memang didefinisikan untuk tipe data tersebut.

- **Symbol Table Management**

Proses pengelolaan symbol table untuk menyimpan dan memeriksa informasi mengenai identifier yang muncul dalam program(apakah identifier yang digunakan telah dideklarasikan, apakah identifier digunakan setelah deklarasi, atau apakah pendeklarasiannya lebih dari sekali).

- **Scope Resolution**

Proses untuk menentukan region akses sebuah variabel berdasarkan hierarki blok kode. Proses ini bertujuan untuk

- **Control Flow Validation**

Proses untuk memastikan bahwa struktur kontrol digunakan sesuai dengan aturan bahasa pemrograman.

Contoh: perintah goto tidak boleh melompat ke bagian program yang tidak valid atau perintah break hanya boleh digunakan di dalam struktur perulangan atau CASE.

Abstract Syntax Tree (AST)

Abstract Syntax Tree (AST) adalah representasi struktur program dalam bentuk pohon yang lebih abstrak dibandingkan parse tree. Representasi ini hanya

mempertahankan elemen-elemen penting dari program dan menghilangkan simbol terminal atau nonterminal perantara yang hanya dibutuhkan untuk proses parsing sehingga lebih menekankan hubungan semantik antarbagian program. Oleh karena itu, desain node-node AST harus mampu merepresentasikan seluruh konstruksi penting dalam bahasa pemrograman, tetapi tetap sederhana dengan menghindari pembentukan node perantara yang tidak diperlukan.

Decorated Abstract Syntax Tree (Decorated AST)

Decorated AST merupakan AST yang dilengkapi informasi semantik setelah melalui tahap semantic analysis. Struktur ini merepresentasikan hasil analisis syntax tree dengan symbol table kemudian diberi anotasi atau informasi tambahan seperti tipe data, deklarasi identifier, dan informasi lain yang diperlukan untuk tahap intermediate code generation.

L-Attributed Grammar

L-Attributed Grammar adalah attribute grammar yang memungkinkan atribut dievaluasi secara top-down dari kiri ke kanan. Pada grammar ini, inherited attribute dari suatu simbol hanya boleh bergantung pada atribut induk atau simbol-simbol di sebelah kirinya, sedangkan synthesized attribute dihitung dari atribut simbol-simbol penyusunnya. Pembatasan ini mengakibatkan proses evaluasi atribut dapat dilakukan secara efisien. Dalam semantic analysis, L-Attributed Grammar digunakan untuk mendukung syntax-directed translation, pembangunan AST, pengisian symbol table, pengecekan tipe, dan pemberian anotasi semantik pada AST.

Perancangan & Implementasi

Perancangan

A. Rancangan Arsitektur

Program pada Milestone 3 ini dirancang sebagai lanjutan dari syntax analyzer pada Milestone 2 dengan menambahkan AST (Abstract Syntax Tree). Seperti milestone sebelumnya, arsitektur menggunakan bahasa C dengan kompilasi C11 melalui gcc.

Program tetap disusun secara modular di dalam folder src. Selain modul yang sudah ada sebelumnya (lexer, token, parser, parse_tree, grammar, fileio), pada Milestone 3 ditambahkan dua modul baru. Pertama, modul ast yang terdiri dari ast_node untuk mendefinisikan struktur node AST beserta operasi dasarnya, serta ast_builder untuk melakukan konversi parse tree menjadi AST menggunakan Syntax-Directed Translation. Kedua, modul semantic yang terdiri dari symbol_table untuk mengelola tabel simbol (tab, btab, atab), serta ast_decorator yang mengimplementasikan visit functions untuk melakukan type checking, scope checking, dan anotasi node AST.

Untuk Milestone 3, komponen terpenting adalah ast_builder.c, symbol_table.c, dan ast_decorator.c. ast_builder.c mengimplementasikan Syntax-Directed Translation melalui fungsi-fungsi buildProgram, buildDeclarationPart, buildStatement, buildExpression, dan fungsi build lain yang masing-masing merepresentasikan satu aturan produksi grammar. symbol_table.c menyediakan antarmuka pengelolaan tabel simbol melalui symInit, symEnter, symLookup, symEnterScope, symExitScope, dan symEnterArray. ast_decorator.c mengimplementasikan visit functions seperti visitDeclPart, visitStatement, visitExpression, visitAssign, visitIf, visitWhile, visitFor, visitRepeat, visitCase, dan visitProcDecl yang masing-masing menangani satu jenis node AST. Alur kode semantic analysis adalah sebagai berikut :

- **Inisialisasi symbol table**

Proses semantic analysis dimulai dengan membaca parse tree dengan memarsing representasi teks ASCII tree yang dihasilkan di Milestone 2.

- **Inisialisasi Symbol Table**

Proses semantic analysis dimulai dengan memanggil symInit yang menginisialisasi ketiga tabel simbol ke kondisi awal. Fungsi ini mengisi indeks 0 hingga 32 pada tab dengan 32 reserved words bahasa Arion. Selanjutnya, indeks 33 hingga 38 diisi dengan 6 predefined identifier. Tabel btab diinisialisasi dengan satu blok global pada indeks 0, dan array display diinisialisasi dengan display[0] = 0 untuk menandai blok aktif pada level 0. Identifier program dan deklarasi pengguna akan dimulai dari indeks 39 dan seterusnya.

- **Pembangunan AST dari Parse Tree**

Setelah itu, fungsi buildAst dipanggil dengan menerima root parse tree sebagai input. Untuk setiap node parse tree, fungsi build yang dipanggil akan menelusuri children, buang terminal yang tidak diperlukan, lalu membangun node baru untuk AST.

- **Traversal AST dan Dekorasi**

Setelah AST terbentuk, fungsi decorateAst dipanggil yang memulai traversal dari root melalui fungsi visit. Traversal menggunakan Depth-First Search (DFS) menggunakan fungsi visit yang saling memanggil secara rekursif sesuai jenis node yang ditemui.

- **Pengelolaan Scope**

Scope dikelola dengan membuat blok baru di symtab tiap masuk prosedur dan fungsi, serta menutupnya saat keluar. Pencarian identifier dilakukan dari scope paling dalam ke scope paling luar.

- **Penanganan Kesalahan**

Jika ada pelanggaran semantik, program mencatat pesan error dan menngghentikan proses dekorasi AST. Contoh error yang dapat didetect

seperti penggunaan identifier yang tidak dikenali, redefinisi identifier pada scope yang sama, serta ketidakcocokan tipe pada assignment dan pemanggilan fungsi.

- Pencetakan Output

Hasil akhir berupa Decorated AST dan isi ketiga table simbol (tab, btab, atab) dicetak ke terminal dan disimpan ke file output.

B. Production rule

1. Rancangan AST Builder

AST Builder mengimplementasikan Syntax-Directed Translation dengan L-Attributed Grammar. Parse tree ditelusuri secara Top-Down, lalu setiap node parse tree dipetakan ke fungsi build yang menghasilkan node AST. Terminal yang tidak memiliki makna semantik dibuang sehingga AST yang dihasilkan hanya berisi hubungan operator dan operand-nya.

Production	Function	Penjelasan
program = program-header declaration-part compound-statement period	buildProgram	Membentuk AST_PROGRAM dengan children AST_DECL_PART dan AST_COMPOUND. Terminal PROGRAMSY, SEMICOLON, PERIOD dibuang.
program-header = programsy ident semicolon	buildProgramHeader	Mengekstrak lexeme IDENT sebagai nama program (synthesized ke atas). Terminal PROGRAMSY, SEMICOLON dibuang.
declaration-part = const-declaration* type-declaration* var-declaration* subprogram-declaration*	buildDeclarationPart	Memanggil dispatchNode untuk setiap child dan mengumpulkan hasilnya ke dalam satu node AST_DECL_PART.
const-declaration = constsy (ident eql constant semicolon)+	buildConstDeclaration	Untuk setiap pasangan ident-constant, membentuk AST_CONST_DECL dengan sval = nama ident dan child = node literal. CONSTSY, EQL, SEMICOLON dibuang.
type-declaration = typesy (ident eql type semicolon)+	buildTypeDeclaration	Untuk setiap pasangan ident-type, membentuk AST_TYPE_DECL dengan sval = nama ident dan child =

		node tipe. TYPESY, EQL, SEMICOLON dibuang.
var-declaration = varsy (identifier-list colon type semicolon)+	buildVarDeclaration	Untuk setiap identifier dalam identifier-list, membentuk satu AST_VAR_DECL dengan sval = nama variabel dan child = node tipe hasil deep copy. VARSY, COLON, SEMICOLON dibuang.
identifier-list = ident (comma ident)*	buildIdentifierList	Mengumpulkan semua lexeme ident ke dalam AST_IDENT_LIST. COMMA dibuang.
type = ident array-type range enumerated record-type	buildType	Meneruskan ke buildArrayType, buildRange, buildEnumerated, buildRecordType, atau membentuk AST_TYPE_IDENT dengan sval = nama tipe sesuai token awal.
array-type = arraysy lbrack (range ident) rbrack ofsy type	buildArrayType	Membentuk AST_ARRAY_TYPE dengan child node index (AST_RANGE atau AST_TYPE_IDENT) dan child node tipe elemen. ARRAYSY, LBRACK, RBRACK, OFSY dibuang.
range = constant period period constant	buildRange	Membentuk AST_RANGE dengan dua child node konstanta sebagai batas bawah dan batas atas. Kedua PERIOD dibuang.
enumerated = lparent ident (comma ident)* rparent	buildEnumerated	Membentuk AST_ENUMERATED dengan setiap ident sebagai child AST_VAR. LPARENT, COMMA, RPARENT dibuang.
record-type = recordsy field-list endsy	buildRecordType	Membentuk AST_RECORD_TYPE dengan setiap field-part sebagai child. RECORDSY, ENDSY dibuang.
field-part = identifier-list colon type	buildFieldPart	Membentuk AST_FIELD_PART dengan child AST_IDENT_LIST dan

		child node tipe. COLON dibuang.
subprogram-declaration = procedure-declaration function-declaration	buildSubprogramDeclaration	Meneruskan ke buildProcedureDeclaration atau buildFunctionDeclaration sesuai token awal.
procedure-declaration = proceduresy ident formal-parameter-list? semicolon block semicolon	buildProcedureDeclaration	Membentuk AST_PROC_DECL dengan sval = nama prosedur, child AST_PARAM_LIST (jika ada), dan child AST_BLOCK. PROCEDURESY, SEMICOLON dibuang.
function-declaration = functionsy ident formal-parameter-list? colon ident semicolon block semicolon	buildFunctionDeclaration	Membentuk AST_FUNC_DECL dengan sval = nama fungsi, child AST_TYPE_IDENT sebagai tipe kembali, child AST_PARAM_LIST (jika ada), dan child AST_BLOCK. FUNCTIONSY, COLON, SEMICOLON dibuang.
block = declaration-part compound-statement	buildBlock	Membentuk AST_BLOCK dengan child AST_DECL_PART dan child AST_COMPOUND.
formal-parameter-list = lparent parameter-group (semicolon parameter-group)* rparent	buildFormalParameterList	Membentuk AST_PARAM_LIST berisi setiap parameter-group sebagai child. LPARENT, SEMICOLON, RPARENT dibuang.
parameter-group = identifier-list colon (ident array-type)	buildParameterGroup	Membentuk AST_PARAM_GROUP dengan child AST_IDENT_LIST dan child node tipe parameter. COLON dibuang.
compound-statement = beginsy statement-list endsy	buildCompoundStatement	Membentuk AST_COMPOUND berisi setiap statement sebagai child. BEGINSY, ENDSY dibuang.
statement-list = statement (semicolon statement)*	buildStatementList	Mengumpulkan setiap node statement ke dalam AST_STMT_LIST. SEMICOLON dibuang. Statement kosong

		menghasilkan AST_EMPTY_STMT.
statement = assignment if while for repeat case call ϵ	buildStatement	Meneruskan ke fungsi build yang sesuai berdasarkan jenis child node. Statement kosong (ϵ) menghasilkan AST_EMPTY_STMT.
variable = ident component-variable*	buildVariable	Membentuk AST_VAR. Akses array ([...]) dibungkus menjadi AST_ARRAY_ACCESS. Akses field (.ident) dibungkus menjadi AST_FIELD_ACCESS.
assignment-statement = variable becomes expression	buildAssignmentStatement	Membentuk AST_ASSIGN dengan child target (hasil buildVariable) dan child value (hasil buildExpression). BECOMES dibuang.
if-statement = ifsy expression thensy statement (elsesy statement)?	buildIfStatement	Membentuk AST_IF dengan child[0] = kondisi, child[1] = branch then, child[2] = branch else jika ada. IFSY, THENSY, ELSESY dibuang.
while-statement = whilesy expression dosy compound-statement semicolon	buildWhileStatement	Membentuk AST_WHILE dengan child[0] = kondisi dan child[1] = body compound. WHILESY, DOSY, SEMICOLON dibuang.
for-statement = forsy ident becomes expression (tosy downtosy) expression dosy compound-statement semicolon	buildForStatement	Membentuk AST_FOR dengan sval = nama counter, sval arah (to/downto), child ekspresi awal, child ekspresi akhir, dan child body compound. FORSY, BECOMES, TOSY/DOWNTOSY, DOSY, SEMICOLON dibuang.
repeat-statement = repeatsy statement-list untilsy expression	buildRepeatStatement	Membentuk AST_REPEAT dengan child AST_STMT_LIST dan child kondisi. REPEATSY, UNTILSY dibuang.
case-statement = casesy expression ofsy case-block+ endsy	buildCaseStatement	Membentuk AST_CASE dengan child[0] = selector dan setiap arm sebagai child AST_CASE_BLOCK. CASESY, OFSY, ENDSY dibuang.

case-block = constant (comma constant)* colon statement	buildCaseBlock	Membentuk AST_CASE_BLOCK dengan setiap konstanta label dan child statement body. COMMA, COLON dibuang.
procedure/function-call = ident (lparent parameter-list? rparent)?	buildProcFuncCall	Membentuk AST_PROC_CALL atau AST_FUNC_CALL dengan sval = nama dan child AST_PARAM_LIST jika ada argumen. LPARENT, RPARENT dibuang.
parameter-list = expression (comma expression)*	buildParameterList	Mengumpulkan setiap ekspresi argumen ke dalam AST_PARAM_LIST. COMMA dibuang.
expression = simple-expression (relational-operator simple-expression)?	buildExpression	Jika ada operator relasional, membentuk AST_BINOP dengan sval = operator. Jika tidak ada, node simple-expression diteruskan langsung (pass-through).
simple-expression = (plus minus)? term (additive-operator term)*	buildSimpleExpression	Unary minus membentuk AST_UNOP. Setiap pasangan (operator, term) melipatkan hasil sebelumnya menjadi AST_BINOP kiri-asosiasi.
term = factor (multiplicative-operator factor)*	buildTerm	Setiap pasangan (operator, factor) melipatkan hasil sebelumnya menjadi AST_BINOP kiri-asosiasi.
factor = ident intcon realcon charcon string lparent expression rparent notsy factor variable	buildFactor	INTCON → AST_INT_LIT. REALCON → AST_REAL_LIT. CHARCON → AST_CHAR_LIT. STRING → AST_STRING_LIT. IDENT → AST_VAR. Kurung → ekspresi dalam diteruskan langsung, kurung dibuang. NOTSY → AST_UNOP dengan sval = "not".

2. Rancangan Semantic Action

Semantic action dijalankan oleh AST Decorator saat menelusuri AST menggunakan visit functions. Setiap visit function bertanggung jawab melakukan pengecekan semantik, memperbarui symbol table, dan menganotasi node dengan informasi tipe, indeks tab, serta lexical level.

Berikut merupakan tabel yang memetakan setiap production rule dengan fungsi visit dan semantic action yang dijalankan.

Production	Function	Penjelasan
program = program-header declaration-part compound-statement period	visitProgram	Mendaftarkan nama program ke tab dengan OBJ_PROGRAM. Menginisialisasi btab[0] sebagai blok global. Meneruskan traversal ke declaration-part dan compound-statement.
const-declaration = constsy (ident eql constant semicolon)+	visitConstDecl	Menentukan tipe konstanta dari node literal. Mengekstrak nilai aktual konstanta dan menyimpannya ke adr. Mendaftarkan identifier ke tab dengan OBJ_CONSTANT.
type-declaration = typesy (ident eql type semicolon)+	visitTypeDecl	Meresolvasi tipe via resolveTypeInfo. Mendaftarkan identifier ke tab dengan OBJ_TYPE. Jika tipe memiliki subrange, menyimpan informasi rangeLow dan rangeHigh ke tab.
var-declaration = varsy (identifier-list colon type semicolon)+	visitVarDecl	Meresolvasi tipe via resolveTypeInfo. Mendaftarkan setiap identifier ke tab dengan OBJ_VARIABLE dan adr = offset pada stack frame. Jika tipe memiliki subrange, menyimpan informasi range ke tab.
procedure-declaration = proceduresy ident formal-parameter-list? semicolon block semicolon	visitProcDecl	Mendaftarkan nama prosedur ke tab dengan OBJ_PROCEDURE. Membuka scope baru, mendaftarkan parameter formal, menelusuri block, lalu menutup scope.
function-declaration = functionsy ident formal-parameter-list? colon ident semicolon block semicolon	visitFuncDecl	Mendaftarkan nama fungsi ke tab dengan OBJ_FUNCTION beserta tipe kembali. Membuat variabel return di dalam scope fungsi. Membuka scope baru, menelusuri block, lalu menutup scope.

range = constant period period constant	resolveTypeInfo (AST_RANGE)	Mengevaluasi kedua batas secara konkret. Memastikan tipe batas bukan Real, kedua batas memiliki tipe yang sama, dan lower bound tidak lebih besar dari upper bound. Mengembalikan TypeInfo dengan informasi low dan high.
enumerated = lparent ident (comma ident)* rparent	resolveTypeInfo (AST_ENUMERATED)	Mendaftarkan setiap identifier ke tab sebagai OBJ_CONSTANT bertipe INTEGER dengan nilai ordinalnya. Menganotasi setiap node ident dengan tabldx, typeldx, dan lexLevel.
array-type = arraysy lbrack (range ident) rbrack ofsy type	resolveTypeInfo (AST_ARRAY_TYPE)	Meresolvasi index type dan memastikan bukan tipe yang tidak valid (Real, Array, Record, dll.). Mendaftarkan array ke atab. Mengembalikan TypeInfo bertipe ARRAY dengan ref ke atab.
record-type = recordsy field-list endsy	resolveTypeInfo (AST_RECORD_TYPE)	Membuat blok baru di btab. Mendaftarkan setiap field ke tab dengan offset-nya masing-masing. Mengembalikan TypeInfo bertipe RECORD dengan ref ke btab.
variable = ident	visitVarRef	Melakukan symLookup pada nama identifier. Memastikan identifier ditemukan, bertipe OBJ_VARIABLE atau OBJ_CONSTANT, dan sudah diinisialisasi jika digunakan sebagai nilai. Menganotasi node dengan tabldx, typeldx, dan lexLevel.
variable = ident lbrack index-list rbrack	visitArrayAccess	Memverifikasi base bertipe ARRAY. Memeriksa tipe index dan memverifikasi nilai literal tidak di luar batas. Menganotasi node dengan typeldx = tipe elemen.
variable = ident period ident	visitFieldAccess	Memverifikasi base bertipe RECORD. Mencari nama field pada record. Mendukung akses field berantai. Menganotasi node

		dengan tabldx dan typeldx field yang ditemukan.
assignment-statement = variable becomes expression	visitAssign	Menghitung tipe target dan tipe value. Memverifikasi kompatibilitas keduanya (assignmentCompatible untuk tipe sederhana, kesamaan tipe dan ref untuk tipe komposit). Menandai variabel target sebagai initialized setelah assignment berhasil.
if-statement = ifsy expression thensy statement (elsesy statement)?	visitIf	Memverifikasi kondisi bertipe BOOLEAN. Meneruskan traversal ke branch then dan else.
while-statement = whilesy expression dosy compound-statement semicolon	visitWhile	Memverifikasi kondisi bertipe BOOLEAN. Meneruskan traversal ke body compound.
for-statement = forsy ident becomes expression (tosy downtosy) expression dosy compound-statement semicolon	visitFor	Memverifikasi counter adalah OBJ_VARIABLE. Memverifikasi tipe batas kompatibel dengan tipe counter. Menandai counter sebagai initialized. Meneruskan traversal ke body.
repeat-statement = repeatsy statement-list untilsy expression	visitRepeat	Meneruskan traversal ke body statement-list. Memverifikasi kondisi until bertipe BOOLEAN.
case-statement = casesy expression ofsy case-block+ endsy	visitCase	Memverifikasi tipe setiap label arm kompatibel dengan tipe selector. Meneruskan traversal ke statement tiap arm.
procedure/function-call = ident (lparent parameter-list? rparent)?	visitCall	Memverifikasi identifier adalah OBJ_PROCEDURE atau OBJ_FUNCTION. Memverifikasi jumlah argumen aktual sesuai. Memverifikasi tipe tiap argumen kompatibel dengan tipe parameter formal.
expression = simple-expression (relational-operator simple-expression)?	visitBinOp	Operator relasional memverifikasi kedua operand kompatibel (INTEGER dan REAL dianggap kompatibel

		satu sama lain); hasil BOOLEAN. Operator aritmatika plus, minus, times: memverifikasi kedua operand numerik (INTEGER atau REAL); hasil REAL jika salah satu operand REAL, INTEGER jika keduanya INTEGER. Operator rdiv (/): memverifikasi kedua operand numerik; hasil selalu REAL. Operator idiv dan imod: memverifikasi kedua operand INTEGER; hasil INTEGER. Operator and, or: memverifikasi kedua operand BOOLEAN; hasil BOOLEAN.
factor = notsy factor / simple-expression = (plus minus)? term	visitUnOp	Operator not: memverifikasi operand BOOLEAN; hasil BOOLEAN. Operator unary minus: memverifikasi operand INTEGER atau REAL; hasil mengikuti tipe operand (INTEGER → INTEGER, REAL → REAL).

Implementasi

AST_Builder

AST_Builder adalah komponen file yang berfungsi untuk membangun tree awal AST dari input hasil parse tree dari parsing source code. Tree awal ini menyimpan struktur sederhana dari parse tree yang menyimpan elemen dan informasi semantik yang penting untuk dipertahankan. Tree awal ini juga akan digunakan untuk pemenuhan entri tabel simbol dan pembuatan decorated AST setelahnya.

Untuk membentuk AST tersebut, sebuah struktur node AstNode diimplementasikan yang akan menyimpan informasi semantik penting dari masing-masing elemen yang dibaca dari parse tree. Setiap node akan menyimpan informasi terkait tipe node AST dari dirinya, informasi children node (jika ada) dan banyak kapasitasnya, informasi nilai data (jika ada) seperti string atau integer, dan indeks tipe dan entri tabel simbol yang akan diisi pada saat dekorasi AST.

```
typedef enum {
    /* Program structure */
    AST_PROGRAM,          /* program Name; decls;
compound */
    AST_BLOCK,            /* declaration-part +
compound-statement */

    /* Declarations */
    AST_CONST_DECL,       /* const Name == value */
    AST_TYPE_DECL,        /* type Name = <type> */
    AST_VAR_DECL,         /* var name : type */
    AST_PARAM_GROUP,      /* parameter group: name :
type */
    AST_PROC_DECL,        /* procedure Name(params);
block */
    AST_FUNC_DECL,        /* function Name(params):
rettype; block */

    /* Type nodes */
    AST_TYPE_IDENT,       /* named type: Integer,
Boolean, dsb */
    AST_ARRAY_TYPE,       /* array [range] of type */
    AST_RECORD_TYPE,      /* record field-list end */
    AST_RANGE,            /* low .. high */
    AST_ENUMERATED,       /* (id1, id2, ...) */
    AST_FIELD_PART,       /* field-list entry: ids :
type */
```



```

        /* Statements */
        AST_COMPOUND,          /* begin statement-list end
*/
        AST_ASSIGN,           /* var := expr */
        AST_IF,               /* if expr then stmt [else
stmt] */
        AST_WHILE,           /* while expr do compound */
        AST_FOR,             /* for id := expr (to|downto)
expr do compound */
        AST_REPEAT,         /* repeat stmts until expr */
        AST_CASE,           /* case expr of case-blocks
end */
        AST_CASE_BLOCK,      /* const-list : statement */
        AST_PROC_CALL,       /* procedure/function call */
        AST_EMPTY_STMT,      /* empty statement */

        /* Expressions */
        AST_BINOP,           /* left op right */
        AST_UNOP,           /* op operand (unary minus,
not) */
        AST_VAR,            /* variable reference */
        AST_ARRAY_ACCESS,   /* arr[index] */
        AST_FIELD_ACCESS,   /* record.field */
        AST_FUNC_CALL,      /* function call in
expression */
        AST_INT_LIT,        /* integer literal */
        AST_REAL_LIT,       /* real literal */
        AST_CHAR_LIT,       /* char literal */
        AST_STRING_LIT,     /* string literal */
        AST_BOOL_LIT,       /* true / false */

        /* Helpers */
        AST_IDENT_LIST,     /* a, b, c */
        AST_PARAM_LIST,     /* actual parameter list */
        AST_DECL_PART,      /* declaration part wrapper
*/
        AST_STMT_LIST       /* statement list wrapper */
} AstNodeType;

typedef struct AstNode {
    AstNodeType type;

    /* nama identifier */
    char *sval;             /* string/ident/operator
value */
    long ival;             /* integer literal */
    double rval;           /* real literal */

    /* Children */
    struct AstNode **children;

```

```

    size_t childCount;
    size_t childCapacity;

    /* Anotasi semantik (diisi oleh visitor saat
    dekorasi) */
    int typeIdx;           /* indeks tipe di tab (-1 =
    belum diisi) */
    int tabIdx;            /* indeks entri di tab (-1 =
    belum diisi) */
    int lexLevel;          /* lexical level saat node
    ini berada */
} AstNode;

```

Beberapa fungsi helper juga diimplementasikan untuk memudahkan pembuatan tree, seperti fungsi instantiasi node, pembebasan node dan penambahan node ke dalam tree.

```

AstNode *astCreateNode(AstNodeType type) {
    AstNode *node = (AstNode *)calloc(1,
    sizeof(AstNode));
    if (node == NULL) return NULL;
    node->type = type;
    node->typeIdx = -1;
    node->tabIdx = -1;
    node->lexLevel = -1;
    return node;
}

AstNode *astCreateSval(AstNodeType type, const char
*sval) {
    AstNode *node = astCreateNode(type);
    if (node == NULL) return NULL;
    node->sval = cloneStr(sval);
    return node;
}

AstNode *astCreateIval(AstNodeType type, long long ival)
{
    AstNode *node = astCreateNode(type);
    if (node == NULL) return NULL;
    node->ival = ival;
    return node;
}

AstNode *astCreateRval(AstNodeType type, double rval) {
    AstNode *node = astCreateNode(type);
    if (node == NULL) return NULL;
    node->rval = rval;
}

```

```

        return node;
    }

int astAddChild(AstNode *parent, AstNode *child) {
    AstNode **newChildren;
    size_t newCapacity;

    if (parent == NULL || child == NULL) return 0;

    if (parent->childCount == parent->childCapacity) {
        newCapacity = parent->childCapacity == 0 ? 4 :
parent->childCapacity * 2;
        newChildren = (AstNode
**)(realloc(parent->children, newCapacity * sizeof(AstNode
*)));
        if (newChildren == NULL) return 0;
        parent->children = newChildren;
        parent->childCapacity = newCapacity;
    }

    parent->children[parent->childCount++] = child;
    return 1;
}

void astFree(AstNode *node) {
    size_t i;

    if (node == NULL) return;
    for (i = 0; i < node->childCount; i++) {
        astFree(node->children[i]);
    }
    free(node->children);
    free(node->sval);
    free(node);
}

```

Proses pembentukan AST akan dimulai oleh fungsi buildAST yang akan menerima parameter node akar parse tree dan melakukan penelusuran sekaligus menyimpan informasi semantik yang penting untuk masing-masing node. Fungsi buildAST akan meneruskan node akar parse tree pada fungsi dispatchNode yang akan membaca label node dari masing-masing node parse tree dan memberikan fungsi builder node AST yang sesuai dengan label node tersebut.

```

AstNode *buildAst(const ParseTreeNode *parseRoot) {
    if (parseRoot == NULL) return NULL;
    return dispatchNode(parseRoot);
}

```

```

}
static AstNode *dispatchNode(const ParseTreeNode *n) {
    if (n == NULL) return NULL;

    if (labelEq(n, "<program>")) return
buildProgram(n);
    if (labelEq(n, "<program-header>")) return
buildProgramHeader(n);
    if (labelEq(n, "<declaration-part>")) return
buildDeclarationPart(n);
    if (labelEq(n, "<const-declaration>")) return
buildConstDeclaration(n);
    if (labelEq(n, "<type-declaration>")) return
buildTypeDeclaration(n);
    if (labelEq(n, "<var-declaration>")) return
buildVarDeclaration(n);
    if (labelEq(n, "<identifier-list>")) return
buildIdentifierList(n);
    if (labelEq(n, "<type>")) return
buildType(n);
    if (labelEq(n, "<array-type>")) return
buildArrayType(n);
    if (labelEq(n, "<range>")) return
buildRange(n);
    if (labelEq(n, "<enumerated>")) return
buildEnumerated(n);
    if (labelEq(n, "<record-type>")) return
buildRecordType(n);
    if (labelEq(n, "<field-part>")) return
buildFieldPart(n);
    if (labelEq(n, "<subprogram-declaration>")) return
buildSubprogramDeclaration(n);
    if (labelEq(n, "<procedure-declaration>")) return
buildProcedureDeclaration(n);
    if (labelEq(n, "<function-declaration>")) return
buildFunctionDeclaration(n);
    if (labelEq(n, "block")) return
buildBlock(n);
    if (labelEq(n, "<formal-parameter-list>")) return
buildFormalParameterList(n);
    if (labelEq(n, "<parameter-group>")) return
buildParameterGroup(n);
    if (labelEq(n, "<compound-statement>")) return
buildCompoundStatement(n);
    if (labelEq(n, "<statement-list>")) return
buildStatementList(n);
    if (labelEq(n, "<statement>")) return
buildStatement(n);
    if (labelEq(n, "<assignment-statement>")) return
buildAssignmentStatement(n);
    if (labelEq(n, "<if-statement>")) return

```

```

buildIfStatement(n);
    if (labelEq(n, "<while-statement>")) return
buildWhileStatement(n);
    if (labelEq(n, "<for-statement>")) return
buildForStatement(n);
    if (labelEq(n, "<repeat-statement>")) return
buildRepeatStatement(n);
    if (labelEq(n, "<case-statement>")) return
buildCaseStatement(n);
    if (labelEq(n, "<case-block>")) return
buildCaseBlock(n);
    if (labelEq(n, "<procedure/function-call>")) return
buildProcFuncCall(n, AST_PROC_CALL);
    if (labelEq(n, "<parameter-list>")) return
buildParameterList(n);
    if (labelEq(n, "<expression>")) return
buildExpression(n);
    if (labelEq(n, "<simple-expression>")) return
buildSimpleExpression(n);
    if (labelEq(n, "<term>")) return
buildTerm(n);
    if (labelEq(n, "<factor>")) return
buildFactor(n);
    if (labelEq(n, "<variable>")) return
buildVariable(n);
    if (labelEq(n, "<constant>")) return
buildConstant(n);

    if (labelEq(n, "<field-list>")) {
        AstNode *list = astCreateNode(AST_DECL_PART);
        size_t i;
        if (list == NULL) return NULL;
        for (i = 0; i < n->childCount; i++) {
            if (labelEq(n->children[i], "<field-part>"))
            {
                AstNode *fp =
buildFieldPart(n->children[i]);
                if (fp == NULL || !astAddChild(list, fp))
{ astFree(list); return NULL; }
            }
        }
        return list;
    }

    return NULL;
}

```

Fungsi builder tersebut diimplementasikan untuk handle suatu kasus grammar tertentu. Setiap node parse tree memiliki informasi grammar dan

semantik yang berbeda sesuai dengan label dari node tersebut, sehingga fungsi builder diimplementasikan untuk menangani node parse tree sesuai grammar label node tersebut.

Pada bagian deklarasi, ast_builder membentuk node seperti AST_CONST_DECL, AST_TYPE_DECL, AST_VAR_DECL, AST_PROC_DECL, dan AST_FUNC_DECL. Untuk deklarasi variabel, builder membaca daftar identifier dan tipe data, kemudian membuat satu node deklarasi untuk setiap identifier. Jika beberapa variabel memiliki tipe yang sama, tipe tersebut dikloning menggunakan cloneAstNode agar setiap deklarasi memiliki node tipe masing-masing.

```
static AstNode *buildDeclarationPart(const ParseTreeNode
*n) {
    AstNode *part = astCreateNode(AST_DECL_PART);
    size_t i;

    if (part == NULL) return NULL;

    for (i = 0; i < n->childCount; i++) {
        AstNode *child = dispatchNode(n->children[i]);
        if (child != NULL && !astAddChild(part, child)) {
            astFree(part);
            return NULL;
        }
    }

    return part;
}

static AstNode *buildConstDeclaration(const ParseTreeNode
*n) {
    AstNode *constPart = astCreateNode(AST_DECL_PART);
    size_t i = 1;

    if (constPart == NULL) return NULL;

    while (i + 2 < n->childCount) {
        char buf[256];
        AstNode *cd;
        AstNode *val;

        if (!labelPrefix(n->children[i], "IDENT(") &&
            !labelPrefix(n->children[i], "ident(")) {
```

```

        i++;
        continue;
    }

    extractLexeme(n->children[i]->label, buf,
sizeof(buf));
    cd = astCreateSval(AST_CONST_DECL, buf);
    if (cd == NULL) { astFree(constPart); return
NULL; }

    if (i + 2 < n->childCount &&
labelEq(n->children[i+2], "<constant>")) {
        val = buildConstant(n->children[i+2]);
        if (val == NULL || !astAddChild(cd, val)) {
astFree(cd); astFree(constPart); return NULL; }
    }

    if (!astAddChild(constPart, cd)) {
astFree(constPart); return NULL; }
    i += 4;
}
return constPart;
}

static AstNode *buildTypeDeclaration(const ParseTreeNode
*n) {
    AstNode *typePart = astCreateNode(AST_DECL_PART);
    size_t i = 1;

    if (typePart == NULL) return NULL;

    while (i + 2 < n->childCount) {
        char buf[256];
        AstNode *td;
        AstNode *typeNode;

        if (!labelPrefix(n->children[i], "IDENT()") &&
!labelPrefix(n->children[i], "ident()")) {
            i++;
            continue;
        }

        extractLexeme(n->children[i]->label, buf,
sizeof(buf));
        td = astCreateSval(AST_TYPE_DECL, buf);
        if (td == NULL) { astFree(typePart); return NULL;
}

        if (i + 2 < n->childCount &&
labelEq(n->children[i+2], "<type>")) {
            typeNode = buildType(n->children[i+2]);

```

```

        if (typeNode == NULL || !astAddChild(td,
typeNode)) { astFree(td); astFree(typePart); return NULL;
    }

    }

    if (!astAddChild(typePart, td)) {
astFree(typePart); return NULL; }
    i += 4;
    }

    return typePart;
}

static AstNode *buildVarDeclaration(const ParseTreeNode
*n) {
    AstNode *varPart = astCreateNode(AST_DECL_PART);
    size_t i = 1;

    if (varPart == NULL) return NULL;

    while (i < n->childCount) {
        AstNode *idList;
        AstNode *typeNode;
        AstNode *vd;
        size_t j;

        if (!labelEq(n->children[i],
"<identifier-list>")) { i++; continue; }

        idList = buildIdentifierList(n->children[i]);
        if (idList == NULL) { astFree(varPart); return
NULL; }

        typeNode = NULL;
        if (i + 2 < n->childCount &&
labelEq(n->children[i+2], "<type>")) {
            typeNode = buildType(n->children[i+2]);
        }

        for (j = 0; j < idList->childCount; j++) {
            const char *name = idList->children[j]->sval;
            vd = astCreateSval(AST_VAR_DECL, name);
            if (vd == NULL) { astFree(idList);
astFree(typeNode); astFree(varPart); return NULL; }

            if (typeNode != NULL) {
                AstNode *typeCopy =
cloneAstNode(typeNode);
                if (typeCopy == NULL) { astFree(vd);
astFree(idList); astFree(typeNode); astFree(varPart);
return NULL; }

```



```

        if (!astAddChild(vd, typeCopy)) {
astFree(typeCopy); astFree(vd); astFree(idList);
astFree(typeNode); astFree(varPart); return NULL; }
    }

    if (!astAddChild(varPart, vd)) {
astFree(idList); astFree(varPart); return NULL; }
    }

    astFree(typeNode);
    astFree(idList);
    i += 4;
}

return varPart;
}

static AstNode *buildSubprogramDeclaration(const
ParseTreeNode *n) {
    if (n->childCount == 1) return
dispatchNode(n->children[0]);
    return NULL;
}

static AstNode *buildProcedureDeclaration(const
ParseTreeNode *n) {
    AstNode *proc;
    char buf[256];
    size_t i;

    buf[0] = '\0';
    for (i = 0; i < n->childCount; i++) {
        if (labelPrefix(n->children[i], "IDENT(") ||
labelPrefix(n->children[i], "ident(")) {
            extractLexeme(n->children[i]->label, buf,
sizeof(buf));
            break;
        }
    }

    proc = astCreateSval(AST_PROC_DECL, buf);
    if (proc == NULL) return NULL;

    for (i = 0; i < n->childCount; i++) {
        const ParseTreeNode *c = n->children[i];
        if (labelEq(c, "<formal-parameter-list>")) {
            AstNode *params =
buildFormalParameterList(c);
            if (params == NULL || !astAddChild(proc,
params)) { astFree(proc); return NULL; }
        } else if (labelEq(c, "block")) {

```

```

        AstNode *block = buildBlock(c);
        if (block == NULL || !astAddChild(proc,
block)) { astFree(proc); return NULL; }
    }

    return proc;
}

static AstNode *buildFunctionDeclaration(const
ParseTreeNode *n) {
    AstNode *func;
    char nameBuf[256];
    char retBuf[256];
    size_t i;
    int identCount = 0;

    nameBuf[0] = '\0';
    retBuf[0] = '\0';

    for (i = 0; i < n->childCount; i++) {
        const ParseTreeNode *c = n->children[i];
        if (labelPrefix(c, "IDENT(") || labelPrefix(c,
"ident(")) {
            identCount++;
            if (identCount == 1) extractLexeme(c->label,
nameBuf, sizeof(nameBuf));
            else extractLexeme(c->label, retBuf,
sizeof(retBuf));
        }
    }

    func = astCreateSval(AST_FUNC_DECL, nameBuf);
    if (func == NULL) return NULL;

    {
        AstNode *ret = astCreateSval(AST_TYPE_IDENT,
retBuf);
        if (ret == NULL || !astAddChild(func, ret)) {
astFree(func); return NULL; }
    }

    for (i = 0; i < n->childCount; i++) {
        const ParseTreeNode *c = n->children[i];
        if (labelEq(c, "<formal-parameter-list>")) {
            AstNode *params =
buildFormalParameterList(c);
            if (params == NULL || !astAddChild(func,
params)) { astFree(func); return NULL; }
        } else if (labelEq(c, "block")) {
            AstNode *block = buildBlock(c);

```

```

        if (block == NULL || !astAddChild(func,
block)) { astFree(func); return NULL; }
    }

    return func;
}

```

Pada statement, builder membentuk node sesuai jenis perintahnya. Assignment direpresentasikan sebagai AST_ASSIGN dengan dua anak, yaitu variabel tujuan dan ekspresi sumber. Percabangan direpresentasikan sebagai AST_IF, perulangan sebagai AST_WHILE, AST_FOR, atau AST_REPEAT, sedangkan pemanggilan prosedur direpresentasikan sebagai AST_PROC_CALL. Statement kosong direpresentasikan menggunakan AST_EMPTY_STMT.

```

static AstNode *buildCompoundStatement(const
ParseTreeNode *n) {
    AstNode *comp = astCreateNode(AST_COMPOUND);
    size_t i;

    if (comp == NULL) return NULL;

    for (i = 0; i < n->childCount; i++) {
        if (labelEq(n->children[i], "<statement-list>"))
        {
            AstNode *stmts =
buildStatementList(n->children[i]);
            if (stmts == NULL || !astAddChild(comp,
stmts)) { astFree(comp); return NULL; }
        }
    }

    return comp;
}

static AstNode *buildStatementList(const ParseTreeNode
*n) {
    AstNode *list = astCreateNode(AST_STMT_LIST);
    size_t i;

    if (list == NULL) return NULL;

    for (i = 0; i < n->childCount; i++) {
        const ParseTreeNode *c = n->children[i];

        if (labelPrefix(c, "SEMICOLON") || labelPrefix(c,

```

```

"semicolon")) continue;

        if (labelEq(c, "<statement>")) {
            AstNode *stmt = buildStatement(c);
            if (stmt == NULL || !astAddChild(list, stmt))
{ astFree(list); return NULL; }
        }
    }

    return list;
}

static AstNode *buildStatement(const ParseTreeNode *n) {
    if (n->childCount == 0) {
        return astCreateNode(AST_EMPTY_STMT);
    }

    return dispatchNode(n->children[0]);
}

static AstNode *buildAssignmentStatement(const
ParseTreeNode *n) {
    AstNode *assign = astCreateNode(AST_ASSIGN);
    AstNode *varNode = NULL;
    AstNode *exprNode = NULL;
    size_t i;

    if (assign == NULL) return NULL;

    for (i = 0; i < n->childCount; i++) {
        const ParseTreeNode *c = n->children[i];
        if (labelEq(c, "<variable>") && varNode == NULL)
            varNode = buildVariable(c);
        else if (labelEq(c, "<expression>") && exprNode
== NULL)
            exprNode = buildExpression(c);
    }

    if (varNode == NULL || exprNode == NULL ||
!astAddChild(assign, varNode) ||
!astAddChild(assign, exprNode)) {
        astFree(assign);
        return NULL;
    }

    return assign;
}

static AstNode *buildIfStatement(const ParseTreeNode *n)
{
    AstNode *ifNode = astCreateNode(AST_IF);

```

```

size_t i;
int foundThen = 0;
int foundElse = 0;

if (ifNode == NULL) return NULL;

for (i = 0; i < n->childCount; i++) {
    const ParseTreeNode *c = n->children[i];
    if (labelEq(c, "<expression>")) {
        AstNode *cond = buildExpression(c);
        if (cond == NULL || !astAddChild(ifNode,
cond)) { astFree(ifNode); return NULL; }
    } else if (labelPrefix(c, "THENSY") ||
labelPrefix(c, "thensy")) {
        foundThen = 1;
    } else if (labelPrefix(c, "ELSESY") ||
labelPrefix(c, "elsesy")) {
        foundElse = 1;
    } else if (labelEq(c, "<statement>") ||
labelEq(c, "<compound-statement>")) {
        AstNode *stmt = dispatchNode(c);
        if (stmt == NULL || !astAddChild(ifNode,
stmt)) { astFree(ifNode); return NULL; }
    }
}

(void)foundThen;
(void)foundElse;
return ifNode;
}

static AstNode *buildWhileStatement(const ParseTreeNode
*n) {
    AstNode *wh = astCreateNode(AST_WHILE);
    size_t i;

    if (wh == NULL) return NULL;

    for (i = 0; i < n->childCount; i++) {
        const ParseTreeNode *c = n->children[i];
        if (labelEq(c, "<expression>")) {
            AstNode *cond = buildExpression(c);
            if (cond == NULL || !astAddChild(wh, cond)) {
astFree(wh); return NULL; }
        } else if (labelEq(c, "<compound-statement>")) {
            AstNode *body = buildCompoundStatement(c);
            if (body == NULL || !astAddChild(wh, body)) {
astFree(wh); return NULL; }
        }
    }
}

```

```

        return wh;
    }

static AstNode *buildForStatement(const ParseTreeNode *n)
{
    AstNode *forNode;
    char buf[256];
    int exprCount = 0;
    int toDir = 1; /* 1 = to, -1 = downto */
    size_t i;

    buf[0] = '\\0';
    for (i = 0; i < n->childCount; i++) {
        if (labelPrefix(n->children[i], "IDENT(") ||
labelPrefix(n->children[i], "ident(")) {
            extractLexeme(n->children[i]->label, buf,
sizeof(buf));
            break;
        }
    }

    forNode = astCreateSval(AST_FOR, buf);
    if (forNode == NULL) return NULL;

    for (i = 0; i < n->childCount; i++) {
        const ParseTreeNode *c = n->children[i];
        if (labelPrefix(c, "DOWNTOSY") || labelPrefix(c,
"downtosy")) {
            toDir = -1;
        } else if (labelEq(c, "<expression>")) {
            AstNode *expr = buildExpression(c);
            if (expr == NULL || !astAddChild(forNode,
expr)) { astFree(forNode); return NULL; }
            exprCount++;
        } else if (labelEq(c, "<compound-statement>")) {
            AstNode *body = buildCompoundStatement(c);
            if (body == NULL || !astAddChild(forNode,
body)) { astFree(forNode); return NULL; }
        }
    }

    forNode->ival = toDir; /* 1=to, -1=downto */
    (void)exprCount;
    return forNode;
}

static AstNode *buildRepeatStatement(const ParseTreeNode
*n) {
    AstNode *rep = astCreateNode(AST_REPEAT);
    size_t i;

```

```

        if (rep == NULL) return NULL;

        for (i = 0; i < n->childCount; i++) {
            const ParseTreeNode *c = n->children[i];
            if (labelEq(c, "<statement-list>")) {
                AstNode *stmts = buildStatementList(c);
                if (stmts == NULL || !astAddChild(rep,
stmts)) { astFree(rep); return NULL; }
            } else if (labelEq(c, "<expression>")) {
                AstNode *cond = buildExpression(c);
                if (cond == NULL || !astAddChild(rep, cond))
{ astFree(rep); return NULL; }
            }
        }

        return rep;
    }

static AstNode *buildCaseStatement(const ParseTreeNode
*n) {
    AstNode *cas = astCreateNode(AST_CASE);
    size_t i;

    if (cas == NULL) return NULL;

    for (i = 0; i < n->childCount; i++) {
        const ParseTreeNode *c = n->children[i];
        if (labelEq(c, "<expression>")) {
            AstNode *expr = buildExpression(c);
            if (expr == NULL || !astAddChild(cas, expr))
{ astFree(cas); return NULL; }
        } else if (labelEq(c, "<case-block>")) {
            AstNode *cb = buildCaseBlock(c);
            if (cb == NULL || !astAddChild(cas, cb)) {
astFree(cas); return NULL; }
        }
    }

    return cas;
}

```

Pada ekspresi, builder membangun struktur pohon berdasarkan operator dan operand. Ekspresi biner seperti $a + b$ dibuat menjadi node `AST_BINOP` yang menyimpan operator pada `sval`, lalu memiliki dua anak: operand kiri dan operand kanan. Ekspresi unary seperti $-x$ atau `not x` dibuat menjadi `AST_UNOP` dengan satu anak. Literal seperti integer, real, char, string, dan boolean direpresentasikan

sebagai AST_INT_LIT,AST_REAL_LIT, AST_CHAR_LIT, AST_STRING_LIT, dan AST_BOOL_LIT.

```
static AstNode *buildExpression(const ParseTreeNode *n) {
    if (n->childCount == 1) {
        return buildSimpleExpression(n->children[0]);
    }

    if (n->childCount == 3) {
        AstNode *left =
buildSimpleExpression(n->children[0]);
        AstNode *right =
buildSimpleExpression(n->children[2]);
        AstNode *binop;
        const ParseTreeNode *opNode = n->children[1];
        char opStr[32] = "?";

        if (opNode->childCount > 0) {
            strncpy(opStr, opNode->children[0]->label,
sizeof(opStr) - 1);
        }

        binop = astCreateSval(AST_BINOP, opStr);
        if (binop == NULL || left == NULL || right ==
NULL ||
            !astAddChild(binop, left) ||
!astAddChild(binop, right)) {
            astFree(left); astFree(right);
astFree(binop);
            return NULL;
        }

        return binop;
    }

    if (n->childCount > 0) return
dispatchNode(n->children[0]);
    return astCreateNode(AST_EMPTY_STMT);
}

static AstNode *buildSimpleExpression(const ParseTreeNode
*n) {
    size_t i = 0;
    AstNode *result = NULL;
    char unary[8] = "";

    if (n->childCount == 0) return
astCreateNode(AST_EMPTY_STMT);

    if (labelPrefix(n->children[0], "PLUS") ||
```



```

labelPrefix(n->children[0], "plus")) {
    strncpy(unary, "+", sizeof(unary) - 1);
    i = 1;
} else if (labelPrefix(n->children[0], "MINUS") ||
labelPrefix(n->children[0], "minus")) {
    strncpy(unary, "-", sizeof(unary) - 1);
    i = 1;
}

if (i >= n->childCount) return
astCreateNode(AST_EMPTY_STMT);

result = buildTerm(n->children[i]);
i++;

if (unary[0] == '-' && result != NULL) {
    AstNode *unop = astCreateSval(AST_UNOP, "-");
    if (unop == NULL || !astAddChild(unop, result)) {
astFree(result); astFree(unop); return NULL; }
    result = unop;
}

while (i + 1 < n->childCount) {
    AstNode *right = buildTerm(n->children[i+1]);
    AstNode *binop;
    const ParseTreeNode *opNode = n->children[i];
    char opStr[32] = "?";

    if (opNode->childCount > 0) {
        strncpy(opStr, opNode->children[0]->label,
sizeof(opStr) - 1);
    }

    binop = astCreateSval(AST_BINOP, opStr);
    if (binop == NULL || right == NULL ||
        !astAddChild(binop, result) ||
!astAddChild(binop, right)) {
        astFree(result); astFree(right);
astFree(binop);
        return NULL;
    }

    result = binop;
    i += 2;
}

return result;
}

static AstNode *buildTerm(const ParseTreeNode *n) {
    size_t i = 0;

```

```

    AstNode *result;

    if (n->childCount == 0) return
astCreateNode(AST_EMPTY_STMT);

    result = buildFactor(n->children[0]);
    i = 1;

    while (i + 1 < n->childCount) {
        AstNode *right = buildFactor(n->children[i+1]);
        AstNode *binop;
        const ParseTreeNode *opNode = n->children[i];
        char opStr[32] = "?";

        if (opNode->childCount > 0) {
            strncpy(opStr, opNode->children[0]->label,
sizeof(opStr) - 1);
        }

        binop = astCreateSval(AST_BINOP, opStr);
        if (binop == NULL || right == NULL ||
!astAddChild(binop, result) ||
!astAddChild(binop, right)) {
            astFree(result); astFree(right);
astFree(binop);
            return NULL;
        }

        result = binop;
        i += 2;
    }

    return result;
}

static AstNode *buildFactor(const ParseTreeNode *n) {
    /* <factor> -> INTCON | REALCON | CHARCON | STRING |
<variable> |
                                LPARENT <expression> RPARENT | NOTSY
<factor> |
                                <procedure/function-call> */
    char buf[512];

    if (n->childCount == 0) return
astCreateNode(AST_EMPTY_STMT);

    {
        const ParseTreeNode *c = n->children[0];

        if (labelPrefix(c, "INTCON(") || labelPrefix(c,
"intcon(")) {

```

```

        long long v = 0;
        extractLexeme(c->label, buf, sizeof(buf));
        v = atoll(buf);
        return astCreateIval(AST_INT_LIT, v);
    }

    if (labelPrefix(c, "REALCON(") || labelPrefix(c,
"realcon(")) {
        double v = 0.0;
        extractLexeme(c->label, buf, sizeof(buf));
        v = atof(buf);
        return astCreateRval(AST_REAL_LIT, v);
    }

    if (labelPrefix(c, "CHARCON(") || labelPrefix(c,
"charcon(")) {
        extractLexeme(c->label, buf, sizeof(buf));
        return astCreateSval(AST_CHAR_LIT, buf);
    }

    if (labelPrefix(c, "STRING(") || labelPrefix(c,
"string(")) {
        extractLexeme(c->label, buf, sizeof(buf));
        return astCreateSval(AST_STRING_LIT, buf);
    }

    if (labelPrefix(c, "NOTSY") || labelPrefix(c,
"notsy")) {
        AstNode *unop = astCreateSval(AST_UNOP,
"not");
        AstNode *operand = (n->childCount > 1) ?
buildFactor(n->children[1]) : NULL;
        if (unop == NULL || operand == NULL ||
!astAddChild(unop, operand)) {
            astFree(unop); astFree(operand);
            return NULL;
        }
        return unop;
    }

    if (labelEq(c, "<variable>"))
return buildVariable(c);
    if (labelEq(c, "<procedure/function-call>"))
return buildProcFuncCall(c, AST_FUNC_CALL);
    if (labelEq(c, "<expression>"))
return buildExpression(c);

    /* LPARENT <expression> RPARENT */
    if (labelPrefix(c, "LPARENT") || labelPrefix(c,
"lparent")) {
        if (n->childCount >= 2) return

```

```

buildExpression(n->children[1]);
    }
}

return astCreateNode(AST_EMPTY_STMT);
}

```

Variabel juga disederhanakan dari parse tree menjadi bentuk AST yang lebih semantik. Identifier biasa menjadi `AST_VAR`. Jika variabel memiliki akses array, builder membuat `AST_ARRAY_ACCESS` dengan variabel dasar dan indeks sebagai anak-anaknya. Jika variabel mengakses field record, builder membuat `AST_FIELD_ACCESS` yang menyimpan objek record dan nama field.

```

static AstNode *buildVariable(const ParseTreeNode *n) {
    AstNode *var = NULL;
    char buf[256];
    size_t i;

    for (i = 0; i < n->childCount; i++) {
        const ParseTreeNode *c = n->children[i];
        if ((labelPrefix(c, "IDENT(") || labelPrefix(c,
"ident(")) && var == NULL) {
            extractLexeme(c->label, buf, sizeof(buf));
            var = astCreateSval(AST_VAR, buf);
            if (var == NULL) return NULL;
        } else if (labelEq(c, "<component-variable>")) {
            const ParseTreeNode *cv = c;
            if (cv->childCount > 0) {
                if (labelPrefix(cv->children[0],
"LBRACK") || labelPrefix(cv->children[0], "lbrack")) {
                    AstNode *acc =
astCreateNode(AST_ARRAY_ACCESS);
                    size_t j;
                    if (acc == NULL || !astAddChild(acc,
var)) { astFree(acc); return NULL; }
                    var = acc;
                    for (j = 0; j < cv->childCount; j++)
                    {
                        if (labelEq(cv->children[j],
"<index-list>")) {
                            if
(!addIndexListChildren(var, cv->children[j])) return
NULL;
                        }
                    }
                }
            }
        } else if (labelPrefix(cv->children[0],

```

```

"PERIOD") || labelPrefix(cv->children[0], "period")) {
    if (cv->childCount >= 2) {
        AstNode *fa =
astCreateNode(AST_FIELD_ACCESS);
        char fieldBuf[256];
        AstNode *field;

extractLexeme(cv->children[1]->label, fieldBuf,
sizeof(fieldBuf));
        field = astCreateSval(AST_VAR,
fieldBuf);
        if (fa == NULL || field == NULL
||
!astAddChild(fa, var) ||
!astAddChild(fa, field)) {
            astFree(fa); astFree(field);
            return NULL;
        }
        var = fa;
    }
}

return var != NULL ? var :
astCreateNode(AST_EMPTY_STMT);
}

```

Setiap node dari parse tree akan ditelusuri untuk diperiksa label dan informasinya, dan akan dibangun menjadi node AST jika node tersebut memiliki informasi semantik yang penting menggunakan fungsi builder yang sesuai. Keseluruhan proses builder ini akan membantu sebuah Abstract Syntax Tree awalan yang akan digunakan kembali sebagai parameter untuk mengisi entri data tabel simbol, dan membentuk Decorated AST.

Struktur Data Symbol_Table

Symbol table adalah struktur data yang diimplementasikan untuk menyimpan informasi terkait identifier dan node yang berada di dalam AST. Informasi ini digunakan dalam tahap analisis semantik dan digunakan selama pembuatan Decorated AST. Informasi ini akan digunakan oleh AST_Decorator untuk memeriksa apakah sebuah identifier sudah dideklarasikan, apakah terjadi redeklarasikan, dan apakah tipe data sudah sesuai.

Symbol table ini terdiri dari tab, btab, dan atab. Tab adalah struktur data tabel simbol yang akan menyimpan informasi terkait setiap identifier di dalam AST. Tabel simbol ini terdiri dari entri data TabEntry yang akan menyimpan nama identifier, kelas objek identifier (obj), tipe identifier (type), alamat relatif identifier (adr), referensi tambahan (jika ada) menuju array atau function (ref), lexical level identifier (lev), dan informasi tambahan identifier seperti parameter function (nrm). Btab adalah struktur data tabel simbol yang akan menyimpan informasi terkait sebuah blok program, seperti procedure atau function di dalam program. Tabel simbol ini terdiri dari entri data BTabEntry yang akan menyimpan entri terakhir simbol tab pada blok program tersebut (last), blok program induk (lpar), ukuran parameter blok program (psze), dan ukuran variabel lokal pada blok program tersebut (vsze). ATabEntry adalah struktur data tabel simbol yang akan menyimpan informasi khusus untuk tipe array. Tabel simbol ini terdiri dari entri data ATabEntry yang akan menyimpan tipe indeks array (xtyp), tipe elemen array (etyp), referensi tambahan (jika ada) elemen array (eref), batas bawah (low) dan batas atas (high) indeks array, ukuran setiap elemen array (elsz), dan ukuran total dari array (size). Keseluruhan struktur data ini diimplementasikan di dalam kode berikut:

```
typedef enum {
    OBJ_CONSTANT,
    OBJ_VARIABLE,
    OBJ_TYPE,
    OBJ_PROCEDURE,
    OBJ_FUNCTION,
    OBJ_PROGRAM
} ObjClass;

typedef enum {
    TYPE_NONE,
    TYPE_INTEGER,
    TYPE_REAL,
    TYPE_BOOLEAN,
    TYPE_CHAR,
    TYPE_STRING,
    TYPE_ARRAY,
    TYPE_RECORD,
    TYPE_VOID
}
```

```

} BaseType;

typedef struct {
    char *identifier;
    int link;
    ObjClass obj;
    BaseType type;
    int ref;
    int nrm;
    int lev;
    int adr;
    bool hasRange;
    BaseType rangeBase;
    int rangeLow;
    int rangeHigh;
    bool initialized;
} TabEntry;

typedef struct {
    int last;
    int lpar;
    int psze;
    int vsze;
} BtabEntry;

typedef struct {
    BaseType xtyp;
    BaseType etyp;
    int eref;
    int low;
    int high;
    int elsz;
    int size;
} AtabEntry;

```

Selain dari struktur data tersebut, beberapa fungsi helper juga diimplementasikan untuk memudahkan penggunaan tabel simbol dalam implementasi AST_Decorator sekaligus memastikan inisialisasi tabel simbol dengan benar.

Sebuah fungsi helper untuk inisialisasi semua tabel simbol digunakan diimplementasikan ke dalam program untuk memastikan inisialisasi tabel simbol memiliki nilai default yang benar dan tidak ada error yang terjadi selama penggunaan tabel simbol. Fungsi `initializeTabEntry`, `initializeBTabEntry`, `initializeATabEntry` digunakan untuk membuat nilai entri default yang benar,

sedangkan fungsi `symInit` digunakan untuk menginisialisasi keseluruhan simbol tabel dengan nilai entri yang benar.

```
static void initializeTabEntry(TabEntry *entry) {
    entry->identifier = NULL;
    entry->link = -1;
    entry->obj = OBJ_TYPE;
    entry->type = TYPE_NONE;
    entry->ref = -1;
    entry->nrm = 0;
    entry->lev = 0;
    entry->adr = 0;
    entry->hasRange = false;
    entry->rangeBase = TYPE_NONE;
    entry->rangeLow = 0;
    entry->rangeHigh = 0;
    entry->initialized = false;
}

static void initializeBtabEntry(BtabEntry *entry) {
    entry->last = -1;
    entry->lpar = -1;
    entry->psize = 0;
    entry->vsize = 0;
}

static void initializeAtabEntry(AtabEntry *entry) {
    entry->xtyp = TYPE_NONE;
    entry->etyp = TYPE_NONE;
    entry->eref = -1;
    entry->low = 0;
    entry->high = 0;
    entry->elsz = 0;
    entry->size = 0;
}

void symInit(void) {
    static const char *reservedWords[32] = {
        "and", "array", "begin", "case", "const", "div",
        "downto", "do",
        "else", "end", "for", "function", "if", "mod",
        "not", "of",
        "or", "procedure", "program", "record", "repeat",
        "integer",
        "real", "boolean", "char", "string", "then",
        "to", "type", "until",
        "var", "while"
    };

    static const struct {
```



```

        const char *name;
        ObjClass obj;
        BaseType type;
        int nrm;
        int adr;
    } predefinedNames[] = {
        { "true", OBJ_CONSTANT, TYPE_BOOLEAN, 1, 1 },
        { "false", OBJ_CONSTANT, TYPE_BOOLEAN, 1, 0 },
        { "writeln", OBJ_PROCEDURE, TYPE_VOID, -1, 0 },
        { "readln", OBJ_PROCEDURE, TYPE_VOID, -1, 0 },
        { "write", OBJ_PROCEDURE, TYPE_VOID, -1, 0 },
        { "read", OBJ_PROCEDURE, TYPE_VOID, -1, 0 }
    };

    currentLevel = 0;
    tabCount = 0;
    btabCount = 0;
    atabCount = 0;
    predefinedCount = 0;
    recordBlockDepth = 0;

    for (int i = 0; i < MAX_TAB; i++) {
        initializeTabEntry(&tab[i]);
    }

    for (int i = 0; i < MAX_BTAB; i++) {
        initializeBtabEntry(&btab[i]);
    }

    for (int i = 0; i < MAX_ATAB; i++) {
        initializeAtabEntry(&atab[i]);
    }

    for (int i = 0; i < MAX_DISPLAY; i++) {
        display[i] = -1;
    }

    for (int i = 0; i < 32; i++) {
        if (tabCount >= MAX_TAB) {
            break;
        }
        tab[tabCount].identifier =
copyString(reservedWords[i]);
        tab[tabCount].link = -1;
        tab[tabCount].obj = OBJ_TYPE;
        tab[tabCount].type =
predefinedTypeName(reservedWords[i]);
        tab[tabCount].ref = -1;
        tab[tabCount].nrm = 0;
        tab[tabCount].lev = 0;
        tab[tabCount].adr = 0;
    }

```

```

        tab[tabCount].initialized = true;
        tabCount++;
    }

    for (int i = 0; i < (int)(sizeof(predefinedNames) /
sizeof(predefinedNames[0])); i++) {
        if (tabCount >= MAX_TAB) {
            break;
        }

        tab[tabCount].identifier =
copyString(predefinedNames[i].name);
        tab[tabCount].link = -1;
        tab[tabCount].obj = predefinedNames[i].obj;
        tab[tabCount].type = predefinedNames[i].type;
        tab[tabCount].ref = -1;
        tab[tabCount].nrm = predefinedNames[i].nrm;
        tab[tabCount].lev = 0;
        tab[tabCount].adr = predefinedNames[i].adr;
        tab[tabCount].initialized = true;

        tabCount++;
    }
    predefinedCount = tabCount;

    initializeBtabEntry(&btab[0]);
    btab[0].last = -1;
    btab[0].lpar = 0;
    btab[0].psze = 0;
    btab[0].vsze = 0;
    btabCount = 1;
    display[0] = 0;
}

```

Fungsi helper lainnya juga diimplementasikan untuk memudahkan konversi beberapa bagian dari struktur data, ataupun untuk memudahkan penelusuran entri di dalam tabel simbol. Fungsi-fungsi helper seperti `objClassToString`, `baseTypeToString`, dan `sizeofBaseType` diimplementasikan untuk memudahkan keseluruhan inisialisasi simbol tabel ataupun penelusuran simbol tabel, sedangkan fungsi-fungsi helper seperti `symEnter`, `symLookup`, `symEnterArray` diimplementasikan untuk memudahkan penggunaan dan penelusuran simbol tabel pada `AST_Decorater`

```

int symLookup(const char *name) {
    if (name == NULL || name[0] == '\\0') {

```

```

        return 0;
    }

    for (int level = currentLevel; level >= 0; level--) {
        int blockIndex = display[level];
        if (blockIndex < 0 || blockIndex >= btabCount) {
            continue;
        }

        int idx = btab[blockIndex].last;
        while (idx != -1) {
            if (tab[idx].identifier != NULL &&
                strcmp(tab[idx].identifier, name) == 0) {
                return idx;
            }
            idx = tab[idx].link;
        }
    }

    for (int i = 33; i < 39 && i < tabCount; i++) {
        if (tab[i].identifier != NULL &&
            strcmp(tab[i].identifier, name) == 0) {
            return i;
        }
    }

    return -1;
}

int symEnterArray(BaseType xtyp, BaseType etyp, int eref,
int low, int high, int elsz) {
    if (atabCount >= MAX_ATAB || low > high || elsz < 0)
    {
        return -1;
    }

    int idx = atabCount++;
    atab[idx].xtyp = xtyp;
    atab[idx].etyp = etyp;
    atab[idx].eref = eref;
    atab[idx].low = low;
    atab[idx].high = high;
    atab[idx].elsz = elsz;
    atab[idx].size = (high - low + 1) * elsz;
    return idx;
}

int symTabCount(void) {
    return tabCount;
}

```

```

int symBtabCount(void) {
    return btabCount;
}

int symAtabCount(void) {
    return atabCount;
}

const char *objClassToString(ObjClass obj) {
    switch (obj) {
        case OBJ_CONSTANT: return "OBJ_CONSTANT";
        case OBJ_VARIABLE: return "OBJ_VARIABLE";
        case OBJ_TYPE: return "OBJ_TYPE";
        case OBJ_PROCEDURE: return "OBJ_PROCEDURE";
        case OBJ_FUNCTION: return "OBJ_FUNCTION";
        case OBJ_PROGRAM: return "OBJ_PROGRAM";
        default: return "OBJ_UNKNOWN";
    }
}

const char *baseTypeToString(BaseType type) {
    switch (type) {
        case TYPE_NONE: return "TYPE_NONE";
        case TYPE_INTEGER: return "TYPE_INTEGER";
        case TYPE_REAL: return "TYPE_REAL";
        case TYPE_BOOLEAN: return "TYPE_BOOLEAN";
        case TYPE_CHAR: return "TYPE_CHAR";
        case TYPE_STRING: return "TYPE_STRING";
        case TYPE_ARRAY: return "TYPE_ARRAY";
        case TYPE_RECORD: return "TYPE_RECORD";
        case TYPE_VOID: return "TYPE_VOID";
        default: return "TYPE_UNKNOWN";
    }
}

```

Terakhir, sebuah fungsi untuk mengeluarkan output hasil entri data pada masing-masing simbol tabel juga diimplementasikan sebagai bagian dari spesifikasi yang sudah diberikan. Fungsi akan melakukan iterasi pada setiap simbol tabel sebanyak jumlah entri data yang ada di dalam tabel tersebut, dan melakukan output pada seluruh atribut dari masing-masing entri tabel.

```

void symPrint(void) {
    printf("TAB:\n");
    printf(" idx | identifier | link | obj | type | ref |
nrm | lev | adr\n");

    printf("-----+-----+-----+-----+-----+-----+---

```

```

--+-----+-----\n");

    for (int i = 0; i < tabCount; i++) {
        if (tab[i].identifier == NULL) {
            continue;
        }
        printf(" %3d | %10s | %4d | %s | %s | %3d | %3d |
%3d | %3d\n",
            i,
            tab[i].identifier,
            tab[i].link < 0 ? 0 : tab[i].link,
            objClassToString(tab[i].obj),
            baseTypeToString(tab[i].type),
            tab[i].ref < 0 ? 0 : tab[i].ref,
            tab[i].nrm,
            tab[i].lev,
            tab[i].adr);
    }

    printf("\nBTAB:\n");
    printf(" idx | last | lpar | psze | vsze\n");
    printf("-----+-----+-----+-----+-----\n");
    for (int i = 0; i < btabCount; i++) {
        printf(" %3d | %4d | %4d | %4d | %4d\n",
            i,
            btab[i].last < 0 ? 0 : btab[i].last,
            btab[i].lpar < 0 ? 0 : btab[i].lpar,
            btab[i].psze,
            btab[i].vsze);
    }

    printf("\nATAB:\n");
    printf(" idx | xtyp | etyp | eref | low | high | elsz
| size\n");
    printf("-----+-----+-----+-----+-----+-----\n");
    for (int i = 0; i < atabCount; i++) {
        printf(" %3d | %5s | %5s | %4d | %3d | %4d | %4d
| %4d\n",
            i,
            baseTypeToString(atab[i].xtyp),
            baseTypeToString(atab[i].etyp),
            atab[i].eref < 0 ? 0 : atab[i].eref,
            atab[i].low,
            atab[i].high,
            atab[i].elsz,
            atab[i].size);
    }

    printf("\nDISPLAY and LEVEL:\n");
    printf(" currentLevel = %d\n", currentLevel);

```

```

    for (int i = 0; i <= currentLevel; i++) {
        printf(" display[%d] = %d\n", i, display[i]);
    }
}

```

AST_Decorator

Hasil AST Tree dari AST_Builder serta struktur data symbol_table yang sudah diimplementasikan akan digunakan untuk membuat decorated AST. Sesuai dengan alur program yang sudah disediakan di dalam spesifikasi, fungsi AST_Decorator akan menerima hasil tree AST sebagai parameter dan melakukan pengecekan menggunakan tabel simbol untuk menghasilkan sebuah decorated AST.

Fungsi utama program ini terletak pada decorateAst yang akan menerima root node AST sebagai parameter dan mengunjungi masing-masing node untuk menyimpan informasi identifier dari masing-masing node. DecorateAst akan memeriksa terlebih dahulu root node untuk memastikan bahwa root node bukan null dan merupakan node awal AST_PROGRAM. Jika root node sudah valid, decorator ini akan menginisiasikan simbol tabel menggunakan symEnter. Jika berhasil diinisiasikan, maka fungsi akan melakukan pengunjungan terhadap masing-masing child node dan memeriksa tipe node tersebut dan melompat ke fungsi yang sesuai dengan node tersebut seperti visitDeclPart dan visitStatement.

```

bool decorateAst(AstNode *root, char *message, size_t
messageSize) {
    g_message = message;
    g_messageSize = messageSize;
    g_hasError = false;
    if (message != NULL && messageSize > 0) {
        message[0] = '\0';
    }

    visit(root);

    return !g_hasError;
}

static void visit(AstNode *node) {
    if (node == NULL) {

```

```

        semError("AST kosong.");
        return;
    }
    if (node->type != AST_PROGRAM) {
        semError("Root AST harus berupa Program.");
        return;
    }

    int pIdx = symEnter(node->sval, OBJ_PROGRAM,
TYPE_VOID, -1, 1, 0);
    if (pIdx == -1) {
        semError("Redeklarasi identifier program '%s'.",
            node->sval ? node->sval : "?");
        return;
    }
    node->tabIdx = pIdx;
    node->lexLevel = 0;

    for (size_t i = 0; i < node->childCount; i++) {
        AstNode *c = node->children[i];
        if (c->type == AST_DECL_PART) {
            visitDeclPart(c);
        }
        if (g_hasError) {
            return;
        }
    }
    for (size_t i = 0; i < node->childCount; i++) {
        AstNode *c = node->children[i];
        if (c->type == AST_COMPOUND) {
            symEnterScope();
            visitStatement(c);
            symExitScope();
        }
        if (g_hasError) {
            return;
        }
    }
}

```

Setiap dari tipe masing-masing node akan memiliki fungsi pengecekan tersendiri yang diimplementasikan di dalam decorator AST tersebut. Fungsi-fungsi tersebut akan digunakan agar decorator dapat melompat dan melakukan penanganan decorator sekaligus penyimpanan entri simbol tabel sesuai dengan tipe node AST yang sedang ditelusuri. Penyimpanan entri simbol tabel akan menggunakan fungsi-fungsi helper yang sudah diimplementasikan sebelumnya di dalam stuktur

data tabel simbol. Beberapa fungsi helper dan pengecekan yang terdapat dalam AST_decorator adalah:

```
static BaseType visitExpression(AstNode *node) {
    if (node == NULL) {
        return TYPE_NONE;
    }

    switch (node->type) {
        case AST_INT_LIT:
            node->typeIdx = TYPE_INTEGER;
            return TYPE_INTEGER;
        case AST_REAL_LIT:
            node->typeIdx = TYPE_REAL;
            return TYPE_REAL;
        case AST_CHAR_LIT:
            node->typeIdx = TYPE_CHAR;
            return TYPE_CHAR;
        case AST_STRING_LIT:
            node->typeIdx = TYPE_STRING;
            return TYPE_STRING;
        case AST_BOOL_LIT:
            node->typeIdx = TYPE_BOOLEAN;
            return TYPE_BOOLEAN;
        case AST_VAR:
            return visitVarRef(node);
        case AST_ARRAY_ACCESS:
            return visitArrayAccess(node);
        case AST_FIELD_ACCESS:
            return visitFieldAccess(node);
        case AST_FUNC_CALL:
            return visitCall(node);
        case AST_BINOP:
            return visitBinOp(node);
        case AST_UNOP:
            return visitUnOp(node);
        default:
            return TYPE_NONE;
    }
}

static BaseType visitArrayAccessMode(AstNode *node, bool
requireInitialized) {
    if (node->childCount < 1) {
        return TYPE_NONE;
    }
    AstNode *base = node->children[0];
    if (base->type != AST_VAR) {
        return TYPE_NONE;
    }
}
```



```

        BaseType baseType = resolveVarReference(base,
requireInitialized);
        if (baseType == TYPE_NONE) {
            return TYPE_NONE;
        }
        int idx = base->tabIdx;
        node->tabIdx = idx;
        node->lexLevel = tab[idx].lev;

        if (baseType != TYPE_ARRAY) {
            semError("Identifier '%s' bukan array sehingga
tidak dapat diindeks.",
                base->sval ? base->sval : "?");
            return TYPE_NONE;
        }

        int aref = tab[idx].ref;
        BaseType currentType = tab[idx].type;
        for (size_t i = 1; i < node->childCount; i++) {
            if (currentType != TYPE_ARRAY || aref < 0) {
                semError("Jumlah index array '%s' melebihi
dimensi array.",
                    base->sval ? base->sval : "?");
                return TYPE_NONE;
            }
            BaseType it = visitExpression(node->children[i]);
            if (it == TYPE_NONE) {
                return TYPE_NONE;
            }
            if (aref >= 0 && it != atab[aref].xtyp &&
                !((it == TYPE_INTEGER) && (atab[aref].xtyp ==
TYPE_INTEGER))) {
                if (it != atab[aref].xtyp) {
                    semError("Tipe index array '%s' tidak
sesuai.",
                        base->sval ? base->sval : "?");
                    return TYPE_NONE;
                }
            }
            if (atab[aref].low < atab[aref].high) {
                long long value = 0;
                BaseType valueType = TYPE_NONE;
                if (constantValue(node->children[i], &value,
&valueType) &&
                    valueType == atab[aref].xtyp &&
                    (value < atab[aref].low || value >
atab[aref].high)) {
                    semError("Index array '%s' berada di luar
range %d..%d.",
                        base->sval ? base->sval : "?",
                        atab[aref].low,

```

```

        atab[aref].high);
        return TYPE_NONE;
    }
}
currentType = atab[aref].etyp;
aref = atab[aref].eref;
}

node->typeIdx = currentType;
return currentType;
}

static void visitCase(AstNode *node) {
    if (node->childCount < 1) {
        return;
    }
    BaseType sel = visitExpression(node->children[0]);
    if (sel == TYPE_NONE) {
        return;
    }
    for (size_t i = 1; i < node->childCount; i++) {
        AstNode *cb = node->children[i];
        if (cb->type != AST_CASE_BLOCK) {
            continue;
        }
        for (size_t j = 0; j < cb->childCount; j++) {
            AstNode *c = cb->children[j];
            if (c->type == AST_INT_LIT || c->type ==
AST_CHAR_LIT ||
c->type == AST_BOOL_LIT || c->type ==
AST_VAR) {
                BaseType labelType = visitExpression(c);
                if (labelType == TYPE_NONE) {
                    return;
                }
                if (!typesCompatible(sel, labelType)) {
                    semError("Tipe label case tidak
kompatibel dengan ekspresi case.");
                    return;
                }
            }
            else {
                visitStatement(c);
            }
        }
    }
}

static void visitStatement(AstNode *node) {
    if (node == NULL) {
        return;
    }
}

```

```

switch (node->type) {
    case AST_COMPOUND:
    case AST_STMT_LIST:
        node->typeIdx = TYPE_VOID;
        node->lexLevel = currentLevel;
        for (size_t i = 0; i < node->childCount; i++)
        {
            visitStatement(node->children[i]);
        }
        break;
    case AST_ASSIGN:
        visitAssign(node);
        break;
    case AST_IF:
        visitIf(node);
        break;
    case AST_WHILE:
        visitWhile(node);
        break;
    case AST_FOR:
        visitFor(node);
        break;
    case AST_REPEAT:
        visitRepeat(node);
        break;
    case AST_CASE:
        visitCase(node);
        break;
    case AST_PROC_CALL:
        (void)visitCall(node);
        break;
    case AST_EMPTY_STMT:
        break;
    default:
        break;
}
}

```

Setiap penelusuran akan menyimpan informasi identifier dari node AST ke dalam tabel simbol. Entri dari tabel simbol ini akan digunakan untuk menyimpan informasi dekorasi pada AST. Node yang berhasil ditelusuri akan diberikan sebuah informasi indeks entri tabel yang memuat informasi node tersebut. Informasi ini akan digunakan saat output program dikeluarkan.

Untuk memastikan bahwa hasil tree AST juga masih sesuai dengan aturan semantic, sebuah fungsi error diimplementasikan ke dalam AST_Decorator. Fungsi penelusuran AST_Decorator akan mengimplementasikan error checking pada masing-masing pengecekan tipe node, dimana node yang error atau salah akan masuk ke dalam fungsi semError. Fungsi ini akan mengset variabel global g_hasError untuk menandakan bahwa AST sudah memiliki node yang error. Jika g_hasError hidup dan bersifat true, maka setiap penelusuran node selanjutnya akan berhenti dan tidak melanjutkan penelusuran hingga leaf node. Setelah itu, fungsi semEnter juga memberikan output terkait node yang memiliki error.

```
static char *g_message = NULL;
static size_t g_messageSize = 0;
static bool g_hasError = false;

static void semError(const char *format, ...) {
    if (g_hasError) {
        return;
    }
    g_hasError = true;

    if (g_message == NULL || g_messageSize == 0) {
        return;
    }

    va_list args;
    va_start(args, format);
    vsnprintf(g_message, g_messageSize, format, args);
    va_end(args);
}
```

Keseluruhan penelusuran dan informasi yang disimpan digunakan untuk memberikan output terhadap AST yang sudah didekorasi. Jika AST_Decorator sudah berhasil dan AST sudah berubah menjadi Decorated AST, maka hasil AST_Decorator dapat dioutputkan menggunakan fungsi printDecoratedAst. Fungsi ini akan melakukan penelusuran kembali pada hasil decorated AST yang sudah dibuat, dan akan mengeluarkan output berdasarkan tipe node yang diperiksa, serta informasi identifier dari entri tabel sesuai dengan indeks entri tabel yang sudah disimpan di dalam node. Penelusuran ini kembali dilakukan

hingga keseluruhan node, sehingga sebuah output hasil decorated AST dapat dikeluarkan.

```
void printDecoratedAst(const AstNode *node, FILE *stream)
{
    if (node == NULL || stream == NULL) {
        return;
    }
    printDecoratedAstRec(node, stream, 0, 1, "");
}

static void printDecoratedAstRec(const AstNode *node,
FILE *stream,
                                int depth, int isLast,
const char *prefix) {
    char nextPrefix[1024];

    if (node == NULL) {
        return;
    }

    if (depth == 0) {
        printDecoratedNode(node, stream);
        prefix = "";
    } else {
        fprintf(stream, "%s%s", prefix, isLast ? "`-- " :
"|-- ");
        printDecoratedNode(node, stream);
    }

    if ((size_t)snprintf(nextPrefix, sizeof(nextPrefix),
"%s%s",
                                prefix, isLast ? "    " : "|
") >= sizeof(nextPrefix)) {
        return;
    }

    for (size_t i = 0; i < node->childCount; i++) {
        printDecoratedAstRec(node->children[i], stream,
depth + 1,
                                (i + 1 == node->childCount),
nextPrefix);
    }
}

static void printDecoratedNode(const AstNode *node, FILE
*stream) {
    fprintf(stream, "%s", decoratedTypeName(node->type));

    if (node->sval != NULL) {
```

```

        fprintf(stream, "('%s')", node->sval);
    } else if (node->type == AST_INT_LIT) {
        fprintf(stream, "(%lld)", node->ival);
    } else if (node->type == AST_REAL_LIT) {
        fprintf(stream, "(%g)", node->rval);
    }

    if (node->typeIdx >= 0 || node->tabIdx >= 0 ||
node->lexLevel >= 0) {
        int first = 1;
        fprintf(stream, "  -> ");
        if (node->typeIdx >= 0) {
            fprintf(stream, "type:%s",
shortTypeName(node->typeIdx));
            first = 0;
        }
        if (node->tabIdx >= 0) {
            if (!first) fprintf(stream, ", ");
            fprintf(stream, "tab_index:%d",
node->tabIdx);
            first = 0;
        }
        if (node->lexLevel >= 0) {
            if (!first) fprintf(stream, ", ");
            fprintf(stream, "lev:%d", node->lexLevel);
        }
    }

    fprintf(stream, "\n");
}

```

Untuk memudahkan penelusuran, penyimpanan informasi node dan entri tabel, dan memudahkan pembuatan decorated AST secara keseluruhan, beberapa fungsi helper juga ditambahkan ke dalam decorator. Fungsi-fungsi ini dapat berupa fungsi yang akan mereturn tipe node dalam bentuk string (decoratedTypeName), fungsi yang akan mereturn tipe data variabel yang dibaca (shortTypeName). Beberapa fungsi helper yang diimplementasikan adalah:

```

static bool isArithmeticOp(const char *op) {
    return op != NULL && (strcmp(op, "plus") == 0 ||
strcmp(op, "minus") == 0 ||
                        strcmp(op, "times") == 0 ||
strcmp(op, "rdiv") == 0 ||
                        strcmp(op, "idiv") == 0 ||
strcmp(op, "imod") == 0);
}

```

```

static bool isRelationalOp(const char *op) {
    return op != NULL && (strcmp(op, "eq1") == 0 ||
        strcmp(op, "neq") == 0 ||
        strcmp(op, "lss") == 0 ||
        strcmp(op, "leq") == 0 ||
        strcmp(op, "gtr") == 0 ||
        strcmp(op, "geq") == 0);
}

static bool isBooleanOp(const char *op) {
    return op != NULL && (strcmp(op, "andsy") == 0 ||
        strcmp(op, "orsy") == 0);
}

static BaseType baseTypeFromName(const char *name) {
    if (name == NULL) {
        return TYPE_NONE;
    }
    if (strcasecmp(name, "integer") == 0) return
TYPE_INTEGER;
    if (strcasecmp(name, "real") == 0)      return
TYPE_REAL;
    if (strcasecmp(name, "boolean") == 0) return
TYPE_BOOLEAN;
    if (strcasecmp(name, "char") == 0)      return
TYPE_CHAR;
    if (strcasecmp(name, "string") == 0) return
TYPE_STRING;
    return TYPE_NONE;
}

static bool typesCompatible(BaseType a, BaseType b) {
    if (a == TYPE_NONE || b == TYPE_NONE) {
        return false;
    }
    if (a == b) {
        return true;
    }
    if ((a == TYPE_INTEGER || a == TYPE_REAL) &&
        (b == TYPE_INTEGER || b == TYPE_REAL)) {
        return true;
    }
    return false;
}

static bool assignmentCompatible(BaseType t1, BaseType
t2) {
    if (t1 == TYPE_NONE || t2 == TYPE_NONE) {
        return false;
    }
}

```

```

    if (t1 == t2) {
        return true;
    }
    if (t1 == TYPE_REAL && t2 == TYPE_INTEGER) {
        return true;
    }
    return false;
}

static const char *decoratedTypeName(AstNodeType type) {
    switch (type) {
        case AST_PROGRAM:      return "Program";
        case AST_BLOCK:        return "Block";
        case AST_CONST_DECL:    return "ConstDecl";
        case AST_TYPE_DECL:     return "TypeDecl";
        case AST_VAR_DECL:      return "VarDecl";
        case AST_PARAM_GROUP:   return "ParamGroup";
        case AST_PROC_DECL:     return "ProcDecl";
        case AST_FUNC_DECL:     return "FuncDecl";
        case AST_TYPE_IDENT:    return "TypeIdent";
        case AST_ARRAY_TYPE:    return "ArrayType";
        case AST_RECORD_TYPE:   return "RecordType";
        case AST_RANGE:         return "Range";
        case AST_ENUMERATED:    return "Enumerated";
        case AST_FIELD_PART:    return "FieldPart";
        case AST_COMPOUND:      return "Compound";
        case AST_ASSIGN:        return "Assign";
        case AST_IF:            return "If";
        case AST_WHILE:         return "While";
        case AST_FOR:           return "For";
        case AST_REPEAT:        return "Repeat";
        case AST_CASE:          return "Case";
        case AST_CASE_BLOCK:    return "CaseBlock";
        case AST_PROC_CALL:     return "ProcCall";
        case AST_EMPTY_STMT:    return "EmptyStmt";
        case AST_BINOP:         return "BinOp";
        case AST_UNOP:          return "UnOp";
        case AST_VAR:           return "Var";
        case AST_ARRAY_ACCESS:   return "ArrayAccess";
        case AST_FIELD_ACCESS:   return "FieldAccess";
        case AST_FUNC_CALL:     return "FuncCall";
        case AST_INT_LIT:       return "IntLit";
        case AST_REAL_LIT:      return "RealLit";
        case AST_CHAR_LIT:      return "CharLit";
        case AST_STRING_LIT:    return "StringLit";
        case AST_BOOL_LIT:      return "BoolLit";
        case AST_IDENT_LIST:    return "IdentList";
        case AST_PARAM_LIST:    return "ParamList";
        case AST_DECL_PART:     return "DeclPart";
        case AST_STMT_LIST:     return "StmtList";
        default:                return "Unknown";
    }
}

```



```

}

static const char *shortTypeName(int typeCode) {
    switch ((BaseType)typeCode) {
        case TYPE_INTEGER: return "integer";
        case TYPE_REAL:     return "real";
        case TYPE_BOOLEAN:  return "boolean";
        case TYPE_CHAR:     return "char";
        case TYPE_STRING:   return "string";
        case TYPE_ARRAY:    return "array";
        case TYPE_RECORD:   return "record";
        case TYPE_VOID:     return "void";
        default:            return "none";
    }
}

```

Pengujian

Testing 1

Keterangan Input : Input Normal Program Arion

Input
<pre> program ScoreCalculator; var baseScore, bonusScore, finalScore: integer; passed: boolean; begin baseScore := 70; bonusScore := 15; finalScore := baseScore + bonusScore; passed := finalScore >= 75; if passed then begin </pre>

```

        writeln('Student passed with score = ', finalScore);
    end
else
begin
    writeln('Student failed with score = ', finalScore);
end;
end.

```

Output

```

=== Decorated AST ===
Program('ScoreCalculator') -> tab_index:38, lev:0
  |-- DeclPart
  |   |-- DeclPart
  |       |-- VarDecl('baseScore') -> type:integer,
tab_index:39, lev:0
  |       |   |-- TypeIdent('integer')
  |       |-- VarDecl('bonusScore') -> type:integer,
tab_index:40, lev:0
  |       |   |-- TypeIdent('integer')
  |       |-- VarDecl('finalScore') -> type:integer,
tab_index:41, lev:0
  |       |   |-- TypeIdent('integer')
  |       |-- VarDecl('passed') -> type:boolean,
tab_index:42, lev:0
  |       |   |-- TypeIdent('boolean')
  |-- Compound -> type:void, lev:1
  |   |-- StmtList -> type:void, lev:1
  |       |-- Assign -> type:void, tab_index:39, lev:0
  |           |-- Var('baseScore') -> type:integer,
tab_index:39, lev:0
  |           |   |-- IntLit(70) -> type:integer

```

```

        |-- Assign -> type:void, tab_index:40, lev:0
        |   |-- Var('bonusScore') -> type:integer,
tab_index:40, lev:0
        |   `-- IntLit(15) -> type:integer
        |-- Assign -> type:void, tab_index:41, lev:0
        |   |-- Var('finalScore') -> type:integer,
tab_index:41, lev:0
        |   `-- BinOp('plus') -> type:integer
        |       |-- Var('baseScore') -> type:integer,
tab_index:39, lev:0
        |       `-- Var('bonusScore') ->
type:integer, tab_index:40, lev:0
        |-- Assign -> type:void, tab_index:42, lev:0
        |   |-- Var('passed') -> type:boolean,
tab_index:42, lev:0
        |   `-- BinOp('geq') -> type:boolean
        |       |-- Var('finalScore') ->
type:integer, tab_index:41, lev:0
        |       `-- IntLit(75) -> type:integer
        `-- If
            |-- Var('passed') -> type:boolean,
tab_index:42, lev:0
            |-- Compound -> type:void, lev:1
            |   |-- StmtList -> type:void, lev:1
            |   |   |-- ProcCall('writeln') ->
type:void, tab_index:34, lev:0
            |   |   `-- ParamList
            |   |       |-- StringLit('Student
passed with score = ') -> type:string
            |   |       `-- Var('finalScore') ->
type:integer, tab_index:41, lev:0

```

```

        `-- Compound -> type:void, lev:1
            `-- StmtList -> type:void, lev:1
                `-- ProcCall('writeln') ->
type:void, tab_index:34, lev:0
                    `-- ParamList
                        |-- StringLit('Student
failed with score = ') -> type:string
                            `-- Var('finalScore') ->
type:integer, tab_index:41, lev:0

```

=== Symbol Table ===

TAB:

idx	identifier	link	obj	type	ref	nrm	lev	adr
0	and	0	OBJ_TYPE	TYPE_NONE	0			
1	array	0	OBJ_TYPE	TYPE_NONE	0			
2	begin	0	OBJ_TYPE	TYPE_NONE	0			
3	case	0	OBJ_TYPE	TYPE_NONE	0			
4	const	0	OBJ_TYPE	TYPE_NONE	0			
5	div	0	OBJ_TYPE	TYPE_NONE	0			
6	downto	0	OBJ_TYPE	TYPE_NONE	0			
7	do	0	OBJ_TYPE	TYPE_NONE	0			

23		boolean		0		OBJ_TYPE		TYPE_BOOLEAN		0	
0		0		0							
24		char		0		OBJ_TYPE		TYPE_CHAR		0	
0		0		0							
25		string		0		OBJ_TYPE		TYPE_STRING		0	
0		0		0							
26		then		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
27		to		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
28		type		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
29		until		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
30		var		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
31		while		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
32		true		0		OBJ_CONSTANT		TYPE_BOOLEAN			
0		1		0		1					
33		false		0		OBJ_CONSTANT		TYPE_BOOLEAN			
0		1		0		0					
34		writeln		0		OBJ_PROCEDURE		TYPE_VOID		0	
	-1		0		0						
35		readln		0		OBJ_PROCEDURE		TYPE_VOID		0	
	-1		0		0						
36		write		0		OBJ_PROCEDURE		TYPE_VOID		0	
	-1		0		0						
37		read		0		OBJ_PROCEDURE		TYPE_VOID		0	
	-1		0		0						
38		ScoreCalculator		0		OBJ_PROGRAM		TYPE_VOID			

<pre> 0 1 0 0 39 baseScore 38 OBJ_VARIABLE TYPE_INTEGER 0 1 0 0 40 bonusScore 39 OBJ_VARIABLE TYPE_INTEGER 0 1 0 1 41 finalScore 40 OBJ_VARIABLE TYPE_INTEGER 0 1 0 2 42 passed 41 OBJ_VARIABLE TYPE_BOOLEAN 0 1 0 3 BTAB: idx last lpar psze vsze -----+-----+-----+-----+----- 0 42 0 0 4 1 0 0 0 0 ATAB: idx xtyp etyp eref low high elsz size -----+-----+-----+-----+-----+-----+-----+----- (kosong - tidak ada array) currentLevel = 0 display[0] = 0 </pre>	
Penjelasan	Program tersebut mendeklarasikan variabel bertipe integer dan boolean, melakukan assignment nilai, melakukan operasi penjumlahan, serta menggunakan percabangan if-else berdasarkan hasil boolean. Output menunjukkan bahwa program berhasil dianalisis tanpa error; AST berhasil didekorasi dengan informasi tipe data, indeks symbol table, dan level scope; serta berbagai ekspresi berhasil dikenali sebagai tipe datanya, seperti <code>baseScore + bonusScore</code> yang dikenali sebagai integer

	dan finalScore >= 75 yang dikenali sebagai boolean.
--	---

Testing 2

Keterangan Input : Input Predefined Identifier dan Tipe Dasar

Input
<pre> program PredefinedDemo; var age: integer; score: real; passed: boolean; initial: char; message: string; begin age := 20; score := 95.5; passed := true; initial := 'A'; message := 'Semantic OK'; writeln(message, age, score, passed, initial); end.</pre>
Output
<pre> === Decorated AST === Program('PredefinedDemo') -> tab_index:38, lev:0 -- DeclPart -- DeclPart</pre>


```

|      |-- VarDecl('age')  -> type:integer,
tab_index:39, lev:0
|      |  `-- TypeIdent('integer')
|      |-- VarDecl('score')  -> type:real,
tab_index:40, lev:0
|      |  `-- TypeIdent('real')
|      |-- VarDecl('passed')  -> type:boolean,
tab_index:41, lev:0
|      |  `-- TypeIdent('boolean')
|      |-- VarDecl('initial')  -> type:char,
tab_index:42, lev:0
|      |  `-- TypeIdent('char')
|      `-- VarDecl('message')  -> type:string,
tab_index:43, lev:0
|      `-- TypeIdent('string')
|-- Compound  -> type:void, lev:1
    |-- StmtList  -> type:void, lev:1
        |-- Assign  -> type:void, tab_index:39, lev:0
            |  |-- Var('age')  -> type:integer,
tab_index:39, lev:0
            |  `-- IntLit(20)  -> type:integer
        |-- Assign  -> type:void, tab_index:40, lev:0
            |  |-- Var('score')  -> type:real,
tab_index:40, lev:0
            |  `-- RealLit(95.5)  -> type:real
        |-- Assign  -> type:void, tab_index:41, lev:0
            |  |-- Var('passed')  -> type:boolean,
tab_index:41, lev:0
            |  `-- Var('true')  -> type:boolean,
tab_index:32, lev:0
            |-- Assign  -> type:void, tab_index:42, lev:0

```

```

        |    |-- Var('initial')  -> type:char,
tab_index:42, lev:0
        |    `-- CharLit('A')   -> type:char
        |-- Assign  -> type:void, tab_index:43, lev:0
        |    |-- Var('message')  -> type:string,
tab_index:43, lev:0
        |    `-- StringLit('Semantic OK')  ->
type:string
        `-- ProcCall('writeln')  -> type:void,
tab_index:34, lev:0
        `-- ParamList
            |-- Var('message')  -> type:string,
tab_index:43, lev:0
            |-- Var('age')      -> type:integer,
tab_index:39, lev:0
            |-- Var('score')    -> type:real,
tab_index:40, lev:0
            |-- Var('passed')   -> type:boolean,
tab_index:41, lev:0
            `-- Var('initial')  -> type:char,
tab_index:42, lev:0

```

=== Symbol Table ===

TAB:

idx	identifier	link	obj	type	ref	nrm	lev	adr
0	and	0	OBJ_TYPE	TYPE_NONE	0			
1	array	0	OBJ_TYPE	TYPE_NONE	0			

17		procedure		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
18		program		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
19		record		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
20		repeat		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
21		integer		0		OBJ_TYPE		TYPE_INTEGER		0	
0		0		0							
22		real		0		OBJ_TYPE		TYPE_REAL		0	
0		0		0							
23		boolean		0		OBJ_TYPE		TYPE_BOOLEAN		0	
0		0		0							
24		char		0		OBJ_TYPE		TYPE_CHAR		0	
0		0		0							
25		string		0		OBJ_TYPE		TYPE_STRING		0	
0		0		0							
26		then		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
27		to		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
28		type		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
29		until		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
30		var		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
31		while		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
32		true		0		OBJ_CONSTANT		TYPE_BOOLEAN			

0		1		0		1	
33		false		0		OBJ_CONSTANT TYPE_BOOLEAN	
0		1		0		0	
34		writeln		0		OBJ_PROCEDURE TYPE_VOID	0
-1		0		0			
35		readln		0		OBJ_PROCEDURE TYPE_VOID	0
-1		0		0			
36		write		0		OBJ_PROCEDURE TYPE_VOID	0
-1		0		0			
37		read		0		OBJ_PROCEDURE TYPE_VOID	0
-1		0		0			
38		PredefinedDemo		0		OBJ_PROGRAM TYPE_VOID	
0		1		0		0	
39		age		38		OBJ_VARIABLE TYPE_INTEGER	
0		1		0		0	
40		score		39		OBJ_VARIABLE TYPE_REAL	0
1		0		1			
41		passed		40		OBJ_VARIABLE TYPE_BOOLEAN	
0		1		0		2	
42		initial		41		OBJ_VARIABLE TYPE_CHAR	0
1		0		3			
43		message		42		OBJ_VARIABLE TYPE_STRING	
0		1		0		4	
BTAB:							
idx		last		lpar		psze	vsze
-----	+	-----	+	-----	+	-----	+
0		43		0		0	5
1		0		0		0	0
ATAB:							

<pre> idx xtyp etyp eref low high elsz size -----+-----+-----+-----+-----+-----+-----+----- (kosong - tidak ada array) currentLevel = 0 display[0] = 0 </pre>	
Penjelasan	Program tersebut menggunakan tipe dasar integer, real, boolean, char, string, konstanta predefined true, dan prosedur predefined writeln. Output menunjukkan bahwa semantic analyzer mampu mengenali tipe-tipe dasar, memberi anotasi tipe pada variabel dan literal, serta menampilkan entri predefined identifier pada symbol table yang dilengkapi dengan variabel-variabel yang dideklarasikan dalam program.

Testing 3

Keterangan Input : Input Deklarasi dan Akses Array

Input
<pre> program ArrayDemo; var numbers: array[1..5] of integer; i: integer; total: integer; begin i := 1; numbers[i] := 10; numbers[2] := 20; </pre>

```

total := numbers[i] + numbers[2];

writeln('Total = ', total);
end.

```

Output

```

=== Decorated AST ===
Program('ArrayDemo') -> tab_index:38, lev:0
  |-- DeclPart
  |   `-- DeclPart
  |       |-- VarDecl('numbers') -> type:array,
tab_index:39, lev:0
  |       |   `-- ArrayType -> type:array
  |       |       |-- Range -> type:integer
  |       |       |   |-- IntLit(1)
  |       |       |   `-- IntLit(5)
  |       |       `-- TypeIdent('integer')
  |       |-- VarDecl('i') -> type:integer,
tab_index:40, lev:0
  |       |   `-- TypeIdent('integer')
  |       `-- VarDecl('total') -> type:integer,
tab_index:41, lev:0
  |       `-- TypeIdent('integer')
  `-- Compound -> type:void, lev:1
      `-- StmtList -> type:void, lev:1
          |-- Assign -> type:void, tab_index:40, lev:0
          |   |-- Var('i') -> type:integer,
tab_index:40, lev:0
          |   `-- IntLit(1) -> type:integer
          |-- Assign -> type:void, tab_index:39, lev:0
          |   |-- ArrayAccess -> type:integer,

```

```

tab_index:39, lev:0
    |    |    |-- Var('numbers')  -> type:array,
tab_index:39, lev:0
    |    |    `-- Var('i')    -> type:integer,
tab_index:40, lev:0
    |    `-- IntLit(10)  -> type:integer
    |-- Assign  -> type:void, tab_index:39, lev:0
    |    |-- ArrayAccess  -> type:integer,
tab_index:39, lev:0
    |    |    |-- Var('numbers')  -> type:array,
tab_index:39, lev:0
    |    |    `-- IntLit(2)  -> type:integer
    |    `-- IntLit(20)  -> type:integer
    |-- Assign  -> type:void, tab_index:41, lev:0
    |    |-- Var('total')  -> type:integer,
tab_index:41, lev:0
    |    `-- BinOp('plus')  -> type:integer
    |    |-- ArrayAccess  -> type:integer,
tab_index:39, lev:0
    |    |    |    |-- Var('numbers')  -> type:array,
tab_index:39, lev:0
    |    |    |    `-- Var('i')  -> type:integer,
tab_index:40, lev:0
    |    |    `-- ArrayAccess  -> type:integer,
tab_index:39, lev:0
    |    |    |    |-- Var('numbers')  -> type:array,
tab_index:39, lev:0
    |    |    |    `-- IntLit(2)  -> type:integer
    |    |    `-- ProcCall('writeln')  -> type:void,
tab_index:34, lev:0
    `-- ParamList

```



```

                                |-- StringLit('Total = ')  ->
type:string
                                `-- Var('total')  -> type:integer,
tab_index:41, lev:0

=== Symbol Table ===
TAB:
  idx | identifier | link | obj | type | ref | nrm | lev |
  adr
-----+-----+-----+-----+-----+-----+-----+
-----
      0 |      and |      0 | OBJ_TYPE | TYPE_NONE |      0 |
0 | 0 | 0
      1 |     array |      0 | OBJ_TYPE | TYPE_NONE |      0 |
0 | 0 | 0
      2 |     begin |      0 | OBJ_TYPE | TYPE_NONE |      0 |
0 | 0 | 0
      3 |      case |      0 | OBJ_TYPE | TYPE_NONE |      0 |
0 | 0 | 0
      4 |      const |      0 | OBJ_TYPE | TYPE_NONE |      0 |
0 | 0 | 0
      5 |       div |      0 | OBJ_TYPE | TYPE_NONE |      0 |
0 | 0 | 0
      6 |    downto |      0 | OBJ_TYPE | TYPE_NONE |      0 |
0 | 0 | 0
      7 |       do |      0 | OBJ_TYPE | TYPE_NONE |      0 |
0 | 0 | 0
      8 |      else |      0 | OBJ_TYPE | TYPE_NONE |      0 |
0 | 0 | 0
      9 |       end |      0 | OBJ_TYPE | TYPE_NONE |      0 |
0 | 0 | 0

```

10		for		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
11		function		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
12		if		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
13		mod		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
14		not		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
15		of		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
16		or		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
17		procedure		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
18		program		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
19		record		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
20		repeat		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
21		integer		0		OBJ_TYPE		TYPE_INTEGER		0	
0		0		0							
22		real		0		OBJ_TYPE		TYPE_REAL		0	
0		0		0							
23		boolean		0		OBJ_TYPE		TYPE_BOOLEAN		0	
0		0		0							
24		char		0		OBJ_TYPE		TYPE_CHAR		0	
0		0		0							
25		string		0		OBJ_TYPE		TYPE_STRING		0	

41	total	40	OBJ_VARIABLE	TYPE_INTEGER			
0	1	0	6				
BTAB:							
idx	last	lpar	psze	vsze			
-----+-----+-----+-----+-----							
0	41	0	0	7			
1	0	0	0	0			
ATAB:							
idx	xtyp	etyp	eref	low	high	elsz	size
-----+-----+-----+-----+-----+-----+-----+-----							
0	TYPE_INTEGER		TYPE_INTEGER		0	1	5
1	5						
currentLevel = 0							
display[0] = 0							
Penjelasan	Program tersebut mendeklarasikan array dengan indeks 1 hingga 5 yang berisi data bertipe integer. Program lalu mengisi array dengan berbagai elemen dan menggunakannya dalam ekspresi matematika. Output menunjukkan bahwa variabel array dicatat pada tab dengan referensi ke atab. Atab menyimpan informasi batas bawah, batas atas, tipe elemen, ukuran elemen, dan total ukuran array. Akses seperti numbers[i] dan numbers[2] dinotasikan sebagai ekspresi bertipe data integer.						

Testing 4

Keterangan Input : Input Deklarasi Record dan Akses Field

Input

```

program RecordDemo;

type
  Point = record
    x: integer;
    y: integer;
  end;

var
  p: Point;
  sum: integer;

begin
  p.x := 10;
  p.y := 20;
  sum := p.x + p.y;

  writeln('Point sum = ', sum);
end.

```

Output

```

=== Decorated AST ===
Program('RecordDemo') -> tab_index:38, lev:0
  |-- DeclPart
  |   |-- DeclPart
  |   |   `-- TypeDecl('Point') -> type:record,
tab_index:41, lev:0
  |   |   `-- RecordType -> type:record
  |   |       |-- FieldPart
  |   |       |   |-- IdentList
  |   |       |   |   `-- Var('x') -> type:integer,

```

```

tab_index:39, lev:0
    |   |   |   |-- TypeIdent('integer')
    |   |   |-- FieldPart
    |   |   |-- IdentList
    |   |   |-- Var('y') -> type:integer,
tab_index:40, lev:0
    |   |   |-- TypeIdent('integer')
    |   |-- DeclPart
    |   |-- VarDecl('p') -> type:record,
tab_index:42, lev:0
    |   |   |-- TypeIdent('Point') -> type:record,
tab_index:41, lev:0
    |   |-- VarDecl('sum') -> type:integer,
tab_index:43, lev:0
    |   |-- TypeIdent('integer')
    |-- Compound -> type:void, lev:1
        |-- StmtList -> type:void, lev:1
            |-- Assign -> type:void, tab_index:39, lev:0
                |-- FieldAccess -> type:integer,
tab_index:39, lev:0
    |   |   |-- Var('p') -> type:record,
tab_index:42, lev:0
    |   |   |-- Var('x') -> type:integer,
tab_index:39, lev:0
    |   |-- IntLit(10) -> type:integer
    |-- Assign -> type:void, tab_index:40, lev:0
        |-- FieldAccess -> type:integer,
tab_index:40, lev:0
    |   |   |-- Var('p') -> type:record,
tab_index:42, lev:0
    |   |   |-- Var('y') -> type:integer,

```

```

tab_index:40, lev:0
    |    `-- IntLit(20)  -> type:integer
    |-- Assign  -> type:void, tab_index:43, lev:0
    |    |-- Var('sum')  -> type:integer,
tab_index:43, lev:0
    |    `-- BinOp('plus')  -> type:integer
    |        |-- FieldAccess  -> type:integer,
tab_index:39, lev:0
    |        |    |-- Var('p')  -> type:record,
tab_index:42, lev:0
    |        |    `-- Var('x')  -> type:integer,
tab_index:39, lev:0
    |        `-- FieldAccess  -> type:integer,
tab_index:40, lev:0
    |        |-- Var('p')  -> type:record,
tab_index:42, lev:0
    |        `-- Var('y')  -> type:integer,
tab_index:40, lev:0
    `-- ProcCall('writeln')  -> type:void,
tab_index:34, lev:0
    `-- ParamList
        |-- StringLit('Point sum = ')  ->
type:string
        `-- Var('sum')  -> type:integer,
tab_index:43, lev:0

=== Symbol Table ===
TAB:
  idx | identifier | link | obj | type | ref | nrm | lev |
  adr
-----+-----+-----+-----+-----+-----+-----+-----+

```

0		and		0		OBJ_TYPE		TYPE_NONE	
0		0		0					
1		array		0		OBJ_TYPE		TYPE_NONE	
0		0		0					
2		begin		0		OBJ_TYPE		TYPE_NONE	
0		0		0					
3		case		0		OBJ_TYPE		TYPE_NONE	
0		0		0					
4		const		0		OBJ_TYPE		TYPE_NONE	
0		0		0					
5		div		0		OBJ_TYPE		TYPE_NONE	
0		0		0					
6		downto		0		OBJ_TYPE		TYPE_NONE	
0		0		0					
7		do		0		OBJ_TYPE		TYPE_NONE	
0		0		0					
8		else		0		OBJ_TYPE		TYPE_NONE	
0		0		0					
9		end		0		OBJ_TYPE		TYPE_NONE	
0		0		0					
10		for		0		OBJ_TYPE		TYPE_NONE	
0		0		0					
11		function		0		OBJ_TYPE		TYPE_NONE	
0		0		0					
12		if		0		OBJ_TYPE		TYPE_NONE	
0		0		0					
13		mod		0		OBJ_TYPE		TYPE_NONE	
0		0		0					
14		not		0		OBJ_TYPE		TYPE_NONE	
0		0		0					

15		of		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
16		or		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
17		procedure		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
18		program		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
19		record		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
20		repeat		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
21		integer		0		OBJ_TYPE		TYPE_INTEGER		0	
0		0		0							
22		real		0		OBJ_TYPE		TYPE_REAL		0	
0		0		0							
23		boolean		0		OBJ_TYPE		TYPE_BOOLEAN		0	
0		0		0							
24		char		0		OBJ_TYPE		TYPE_CHAR		0	
0		0		0							
25		string		0		OBJ_TYPE		TYPE_STRING		0	
0		0		0							
26		then		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
27		to		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
28		type		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
29		until		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
30		var		0		OBJ_TYPE		TYPE_NONE		0	

0		0		0									
31				while		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0									
32				true		0		OBJ_CONSTANT		TYPE_BOOLEAN			
0		1		0		1							
33				false		0		OBJ_CONSTANT		TYPE_BOOLEAN			
0		1		0		0							
34				writeln		0		OBJ_PROCEDURE		TYPE_VOID		0	
	-1		0		0								
35				readln		0		OBJ_PROCEDURE		TYPE_VOID		0	
	-1		0		0								
36				write		0		OBJ_PROCEDURE		TYPE_VOID		0	
	-1		0		0								
37				read		0		OBJ_PROCEDURE		TYPE_VOID		0	
	-1		0		0								
38		RecordDemo				0		OBJ_PROGRAM		TYPE_VOID		0	
1		0		0									
39				x		0		OBJ_VARIABLE		TYPE_INTEGER			
0		0		0		0							
40				y		39		OBJ_VARIABLE		TYPE_INTEGER			
0		0		0		1							
41				Point		38		OBJ_TYPE		TYPE_RECORD		1	
1		0		0									
42				p		41		OBJ_VARIABLE		TYPE_RECORD			
1		1		0		0							
43				sum		42		OBJ_VARIABLE		TYPE_INTEGER			
0		1		0		2							
BTAB:													
idx last lpar psze vsze													
-----+-----+-----+-----+-----													

<pre> 0 43 0 0 3 1 40 0 0 2 2 0 0 0 0 </pre> ATAB: <pre> idx xtyp etyp eref low high elsz size -----+-----+-----+-----+-----+-----+-----+----- (kosong - tidak ada array) </pre> <pre> currentLevel = 0 display[0] = 0 </pre>	
Penjelasan	Program tersebut mendefinisikan tipe record Point dengan field x dan y. Program kemudian mendeklarasikan variabel yang bertipe Point, lalu mengakses field yang terdapat pada variabel tersebut. Output menunjukkan bahwa tipe record dicatat sebagai tipe terstruktur pada symbol table, field-field record tersimpan sebagai entri yang terhubung dengan block record, dan akses field pada decorated AST berhasil dikenali serta diberi tipe integer.

Testing 5

Keterangan Input : Input Prosedur, Parameter, Variabel Lokal, dan Scope

Input
<pre> program ProcedureDemo; var result: integer; procedure addBonus(base: integer; bonus: integer); </pre>

```

var
  localTotal: integer;
begin
  localTotal := base + bonus;
  result := localTotal;
end;

begin
  result := 0;
  addBonus(70, 15);
  writeln('Result = ', result);
end.

```

Output

```

=== Decorated AST ===
Program('ProcedureDemo') -> tab_index:38, lev:0
  |-- DeclPart
  |   |-- DeclPart
  |   |   `-- VarDecl('result') -> type:integer,
tab_index:39, lev:0
  |   |   `-- TypeIdent('integer')
  |   `-- ProcDecl('addBonus') -> tab_index:40, lev:0
  |       |-- ParamList
  |       |   |-- ParamGroup
  |       |   |   |-- IdentList
  |       |   |   |   `-- Var('base') -> type:integer,
tab_index:41, lev:1
  |       |   |   |   `-- TypeIdent('integer')
  |       |   |   `-- ParamGroup
  |       |   |       |-- IdentList
  |       |   |       |   `-- Var('bonus') -> type:integer,

```

```

tab_index:42, lev:1
  |          |          |-- TypeIdent('integer')
  |          |-- Block
  |          |-- DeclPart
  |          | |-- DeclPart
  |          | |-- VarDecl('localTotal') ->
type:integer, tab_index:43, lev:1
  |          |          |-- TypeIdent('integer')
  |          |-- Compound -> type:void, lev:1
  |          |-- StmtList -> type:void, lev:1
  |          |-- Assign -> type:void,
tab_index:43, lev:1
  |          | |-- Var('localTotal') ->
type:integer, tab_index:43, lev:1
  |          | |-- BinOp('plus') ->
type:integer
  |          | |-- Var('base') ->
type:integer, tab_index:41, lev:1
  |          | |-- Var('bonus') ->
type:integer, tab_index:42, lev:1
  |          |-- Assign -> type:void,
tab_index:39, lev:0
  |          |-- Var('result') ->
type:integer, tab_index:39, lev:0
  |          |-- Var('localTotal') ->
type:integer, tab_index:43, lev:1
  |-- Compound -> type:void, lev:1
    |-- StmtList -> type:void, lev:1
      |-- Assign -> type:void, tab_index:39, lev:0
        |-- Var('result') -> type:integer,
tab_index:39, lev:0

```

```

        |    `-- IntLit(0)  -> type:integer
        |-- ProcCall('addBonus')  -> type:void,
tab_index:40, lev:0
        |    `-- ParamList
        |        |-- IntLit(70)  -> type:integer
        |        `-- IntLit(15)  -> type:integer
        `-- ProcCall('writeln')  -> type:void,
tab_index:34, lev:0
        `-- ParamList
            |-- StringLit('Result = ')  ->
type:string
            `-- Var('result')  -> type:integer,
tab_index:39, lev:0

=== Symbol Table ===
TAB:
  idx | identifier | link | obj | type | ref | nrm | lev |
adr
-----+-----+-----+-----+-----+-----+-----+-----+
-----
      0 |          and |      0 | OBJ_TYPE | TYPE_NONE |      0 |
0 |  0 |  0
      1 |         array |      0 | OBJ_TYPE | TYPE_NONE |      0 |
0 |  0 |  0
      2 |         begin |      0 | OBJ_TYPE | TYPE_NONE |      0 |
0 |  0 |  0
      3 |          case |      0 | OBJ_TYPE | TYPE_NONE |      0 |
0 |  0 |  0
      4 |         const |      0 | OBJ_TYPE | TYPE_NONE |      0 |
0 |  0 |  0
      5 |          div |      0 | OBJ_TYPE | TYPE_NONE |      0 |

```


21		integer		0		OBJ_TYPE		TYPE_INTEGER		0	
0		0		0							
22		real		0		OBJ_TYPE		TYPE_REAL		0	
0		0		0							
23		boolean		0		OBJ_TYPE		TYPE_BOOLEAN		0	
0		0		0							
24		char		0		OBJ_TYPE		TYPE_CHAR		0	
0		0		0							
25		string		0		OBJ_TYPE		TYPE_STRING		0	
0		0		0							
26		then		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
27		to		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
28		type		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
29		until		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
30		var		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
31		while		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
32		true		0		OBJ_CONSTANT		TYPE_BOOLEAN			
0		1		0						1	
33		false		0		OBJ_CONSTANT		TYPE_BOOLEAN			
0		1		0						0	
34		writeln		0		OBJ_PROCEDURE		TYPE_VOID		0	
	-1		0		0						
35		readln		0		OBJ_PROCEDURE		TYPE_VOID		0	
	-1		0		0						
36		write		0		OBJ_PROCEDURE		TYPE_VOID		0	


```

| -1 | 0 | 0
37 | read | 0 | OBJ_PROCEDURE | TYPE_VOID | 0
| -1 | 0 | 0
38 | ProcedureDemo | 0 | OBJ_PROGRAM | TYPE_VOID |
0 | 1 | 0 | 0
39 | result | 38 | OBJ_VARIABLE | TYPE_INTEGER |
0 | 1 | 0 | 0
40 | addBonus | 39 | OBJ_PROCEDURE | TYPE_VOID | 1
| 2 | 0 | 0
41 | base | 0 | OBJ_VARIABLE | TYPE_INTEGER |
0 | 0 | 1 | 0
42 | bonus | 41 | OBJ_VARIABLE | TYPE_INTEGER |
0 | 0 | 1 | 1
43 | localTotal | 42 | OBJ_VARIABLE | TYPE_INTEGER |
0 | 1 | 1 | 0

```

BTAB:

```

idx | last | lpar | psze | vsze
-----+-----+-----+-----+-----
0 | 40 | 0 | 0 | 1
1 | 43 | 42 | 2 | 1
2 | 0 | 0 | 0 | 0

```

ATAB:

```

idx | xtyp | etyp | eref | low | high | elsz | size
-----+-----+-----+-----+-----+-----+-----+-----
(kosong - tidak ada array)

```

currentLevel = 0

display[0] = 0

Penjelasan	Program tersebut mendeklarasikan prosedur addBonus yang memiliki parameter base dan bonus, menggunakan variabel lokal localTotal, lalu mengubah variabel global result dari dalam prosedur. Output menunjukkan bahwa prosedur memiliki entri pada tab, parameter dan variabel lokal dicatat pada block prosedur di btab, serta semantic analyzer mampu melakukan resolusi identifier antar scope, yaitu membedakan parameter, variabel lokal, dan variabel global.
-------------------	---

Testing 6

Keterangan Input : Input Tipe Subrange

Input
<pre> program SubrangeDemo; var index: 1..10; value: integer; begin index := 5; value := index + 10; writeln('Value = ', value); end.</pre>
Output
<pre> === Decorated AST === Program('SubrangeDemo') -> tab_index:38, lev:0 -- DeclPart</pre>

```

|    `-- DeclPart
|        |-- VarDecl('index')  -> type:integer,
tab_index:39, lev:0
|        |    `-- Range  -> type:integer
|        |        |-- IntLit(1)
|        |        `-- IntLit(10)
|        `-- VarDecl('value')  -> type:integer,
tab_index:40, lev:0
|            `-- TypeIdent('integer')
`-- Compound  -> type:void, lev:1
    `-- StmtList  -> type:void, lev:1
        |-- Assign  -> type:void, tab_index:39, lev:0
        |    |-- Var('index')  -> type:integer,
tab_index:39, lev:0
        |    |    `-- IntLit(5)  -> type:integer
        |    |-- Assign  -> type:void, tab_index:40, lev:0
        |    |    |-- Var('value')  -> type:integer,
tab_index:40, lev:0
        |    |    |    `-- BinOp('plus')  -> type:integer
        |    |    |        |-- Var('index')  -> type:integer,
tab_index:39, lev:0
        |    |    |        |    `-- IntLit(10)  -> type:integer
        |    |    |        `-- ProcCall('writeln')  -> type:void,
tab_index:34, lev:0
        |    |    `-- ParamList
        |    |        |-- StringLit('Value = ')  ->
type:string
        |    |        `-- Var('value')  -> type:integer,
tab_index:40, lev:0

=== Symbol Table ===

```

TAB:

idx	identifier	link	obj	type	ref	nrm	lev	adr
-----+	-----+	-----+	-----+	-----+	-----+	-----+	-----+	-----+
0	and	0	OBJ_TYPE	TYPE_NONE	0			
0	0	0						
1	array	0	OBJ_TYPE	TYPE_NONE	0			
0	0	0						
2	begin	0	OBJ_TYPE	TYPE_NONE	0			
0	0	0						
3	case	0	OBJ_TYPE	TYPE_NONE	0			
0	0	0						
4	const	0	OBJ_TYPE	TYPE_NONE	0			
0	0	0						
5	div	0	OBJ_TYPE	TYPE_NONE	0			
0	0	0						
6	downto	0	OBJ_TYPE	TYPE_NONE	0			
0	0	0						
7	do	0	OBJ_TYPE	TYPE_NONE	0			
0	0	0						
8	else	0	OBJ_TYPE	TYPE_NONE	0			
0	0	0						
9	end	0	OBJ_TYPE	TYPE_NONE	0			
0	0	0						
10	for	0	OBJ_TYPE	TYPE_NONE	0			
0	0	0						
11	function	0	OBJ_TYPE	TYPE_NONE	0			
0	0	0						
12	if	0	OBJ_TYPE	TYPE_NONE	0			
0	0	0						

13		mod		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
14		not		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
15		of		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
16		or		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
17		procedure		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
18		program		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
19		record		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
20		repeat		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
21		integer		0		OBJ_TYPE		TYPE_INTEGER		0	
0		0		0							
22		real		0		OBJ_TYPE		TYPE_REAL		0	
0		0		0							
23		boolean		0		OBJ_TYPE		TYPE_BOOLEAN		0	
0		0		0							
24		char		0		OBJ_TYPE		TYPE_CHAR		0	
0		0		0							
25		string		0		OBJ_TYPE		TYPE_STRING		0	
0		0		0							
26		then		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
27		to		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
28		type		0		OBJ_TYPE		TYPE_NONE		0	

0		0		0									
29				until		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0									
30				var		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0									
31				while		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0									
32				true		0		OBJ_CONSTANT		TYPE_BOOLEAN			
0		1		0		1							
33				false		0		OBJ_CONSTANT		TYPE_BOOLEAN			
0		1		0		0							
34				writeln		0		OBJ_PROCEDURE		TYPE_VOID		0	
	-1		0		0								
35				readln		0		OBJ_PROCEDURE		TYPE_VOID		0	
	-1		0		0								
36				write		0		OBJ_PROCEDURE		TYPE_VOID		0	
	-1		0		0								
37				read		0		OBJ_PROCEDURE		TYPE_VOID		0	
	-1		0		0								
38		SubrangeDemo				0		OBJ_PROGRAM		TYPE_VOID		0	
	1		0		0								
39				index		38		OBJ_VARIABLE		TYPE_INTEGER			
0		1		0		0							
40				value		39		OBJ_VARIABLE		TYPE_INTEGER			
0		1		0		1							
BTAB:													
idx last lpar psze vsze													
-----+-----+-----+-----+-----													
0		40		0		0		0		2			
1		0		0		0		0		0			

ATAB: <pre> idx xtyp etyp eref low high elsz size -----+-----+-----+-----+-----+-----+-----+----- (kosong - tidak ada array) currentLevel = 0 display[0] = 0 </pre>	
Penjelasan	Program tersebut mendeklarasikan variabel index dengan tipe subrange 1 hingga 10, memberikan nilai yang berada dalam rentang tersebut, lalu menggunakan variabel tersebut dalam ekspresi aritmatika bersama variabel integer. Output menunjukkan bahwa deklarasi subrange dikenali pada decorated AST melalui node Range. Pada symbol table, subrange tersebut direpresentasikan sebagai tipe dasar integer.

Testing 7

Keterangan Input : Input Deteksi Error Tipe Data

Input
<pre> program SemanticErrorDemo; var total: integer; name: string; begin name := 'Audric'; total := name; end. </pre>

Output

Semantic error: Type mismatch: tidak dapat assign
TYPE_STRING ke TYPE_INTEGER.

=== Decorated AST ===

```
Program('SemanticErrorDemo') -> tab_index:38, lev:0
  |-- DeclPart
  |   `-- DeclPart
  |       |-- VarDecl('total') -> type:integer,
tab_index:39, lev:0
  |       |   `-- TypeIdent('integer')
  |       |-- VarDecl('name') -> type:string,
tab_index:40, lev:0
  |       |   `-- TypeIdent('string')
  |       `-- Compound -> type:void, lev:1
  |           `-- StmtList -> type:void, lev:1
  |               |-- Assign -> type:void, tab_index:40, lev:0
  |               |   |-- Var('name') -> type:string,
tab_index:40, lev:0
  |               |   `-- StringLit('Audric') -> type:string
  |               `-- Assign
  |                   |-- Var('total') -> type:integer,
tab_index:39, lev:0
  |                   `-- Var('name') -> type:string,
tab_index:40, lev:0
```

=== Symbol Table ===

TAB:

idx	identifier	link	obj	type	ref	nrm	lev	adr
-----	------------	------	-----	------	-----	-----	-----	-----

-----+-----+-----+-----+-----+-----+-----+-----+																			

0		and		0		OBJ_TYPE		TYPE_NONE		0									
0		0		0															
1		array		0		OBJ_TYPE		TYPE_NONE		0									
0		0		0															
2		begin		0		OBJ_TYPE		TYPE_NONE		0									
0		0		0															
3		case		0		OBJ_TYPE		TYPE_NONE		0									
0		0		0															
4		const		0		OBJ_TYPE		TYPE_NONE		0									
0		0		0															
5		div		0		OBJ_TYPE		TYPE_NONE		0									
0		0		0															
6		downto		0		OBJ_TYPE		TYPE_NONE		0									
0		0		0															
7		do		0		OBJ_TYPE		TYPE_NONE		0									
0		0		0															
8		else		0		OBJ_TYPE		TYPE_NONE		0									
0		0		0															
9		end		0		OBJ_TYPE		TYPE_NONE		0									
0		0		0															
10		for		0		OBJ_TYPE		TYPE_NONE		0									
0		0		0															
11		function		0		OBJ_TYPE		TYPE_NONE		0									
0		0		0															
12		if		0		OBJ_TYPE		TYPE_NONE		0									
0		0		0															
13		mod		0		OBJ_TYPE		TYPE_NONE		0									
0		0		0															
14		not		0		OBJ_TYPE		TYPE_NONE		0									

30		var		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
31		while		0		OBJ_TYPE		TYPE_NONE		0	
0		0		0							
32		true		0		OBJ_CONSTANT		TYPE_BOOLEAN			
0		1		0		1					
33		false		0		OBJ_CONSTANT		TYPE_BOOLEAN			
0		1		0		0					
34		writeln		0		OBJ_PROCEDURE		TYPE_VOID		0	
	-1		0		0						
35		readln		0		OBJ_PROCEDURE		TYPE_VOID		0	
	-1		0		0						
36		write		0		OBJ_PROCEDURE		TYPE_VOID		0	
	-1		0		0						
37		read		0		OBJ_PROCEDURE		TYPE_VOID		0	
	-1		0		0						
38		SemanticErrorDemo		0		OBJ_PROGRAM		TYPE_VOID			
	0		1		0		0				
39		total		38		OBJ_VARIABLE		TYPE_INTEGER			
0		1		0		0					
40		name		39		OBJ_VARIABLE		TYPE_STRING			
0		1		0		1					
BTAB:											
idx		last		lpar		psze		vsze			
-----	+	-----	+	-----	+	-----	+	-----			
0		40		0		0		2			
1		0		0		0		0			
ATAB:											
idx		xtyp		etyp		eref		low		high	
								elsz		size	

<pre> -----+-----+-----+-----+-----+-----+-----+----- (kosong - tidak ada array) currentLevel = 0 display[0] = 0 </pre>	
Penjelasan	Program tersebut mendeklarasikan variabel total bertipe integer dan name bertipe string, lalu mencoba melakukan assignment nilai string ke variabel integer melalui perintah <code>total := name</code>. Output menunjukkan bahwa semantic analyzer berhasil mendeteksi ketidakcocokan tipe pada assignment dan menghasilkan pesan error.

Kesimpulan dan Saran

Kesimpulan

Program berhasil dirancang dan diimplementasikan dengan baik. Program berhasil melakukan semantic analysis terhadap source code berdasarkan parse tree yang dibentuk. Proses semantic analysis berhasil membangun decorated AST, menyusun symbol table, serta memeriksa kesesuaian atribut semantik. Output yang dihasilkan sudah mengikuti kebutuhan milestone, yaitu penampilan decorated AST dan symbol table yang merepresentasikan informasi semantik program.

Program juga berhasil menunjukkan error ketika bertemu dengan test case yang tidak memenuhi aturan semantik language. Secara keseluruhan, semantic analysis berhasil diimplementasikan ke dalam source code program dengan tetap mengikuti alur L-Attributed Grammar.

Saran

Spesifikasi tugas besar tidak terlalu berat dan tergolong bisa dinikmati pengerjaannya, serta spesifikasi masih sesuai dengan cakupan materi yang diajarkan di kelas.

Harapannya ada konsistensi untuk masalah input di spesifikasi menyatakan input .txt parse tree, tetapi di QnA menyatakan input .txt source code.

Lampiran

Link Repositori

<https://github.com/RayapSunggal/SHT-Tubes-IF2224-2026>

Pembagian Tugas

Nama	Tugas	Persentase
Audric Y. M.Simatupang - 13524010	Testing source code, Pembuatan hasil test case pada laporan, Penyusunan laporan	25%
Raynard Fausta - 13524052	Bugfixing source code	25%
Nathan E.C Marpaung - 13524062	Pembuatan source code symbol_table	25%
Richard S Simanullang - 13524112	Pembuatan source code ast dan ast_decorator	25%

Referensi

- Laboratorium Ilmu Rekayasa dan Komputasi. 2026. Arion Compiler: Milestone 3 - Semantic Analysis. Bandung. Institut Teknologi Bandung.
- Siegfried, R. M. 2004. *Compiler construction: Lecture 8 – Semantic analysis [Lecture slides]*. New York. Adelphi University.
- Pgrandinetti. 2019. *What Is Semantic Analysis in a Compiler?* . Diakses pada 21 Mei 2026 dari <https://pgrandinetti.github.io/compiler/page/what-is-semantic-analysis-in-compilers/>.
- Ankit87. 2026. *Symbol Table in Compiler*. GeeksForGeeks. Diakses pada 21 Mei 2026 dari <https://www.geeksforgeeks.org/compiler-design/symbol-table-compiler/>.
- Singhal, Palak. 2026. *Semantic Analysis in Compiler Design*. GeeksForGeeks. Diakses pada 21 Mei 2026 dari <https://www.geeksforgeeks.org/compiler-design/semantic-analysis-in-compiler-design/>.
- Krolik, Alexander. 2020. *Abstract Syntax Trees*. Montreal. McGill University.
- Zaker, Morteza. Lecture 8: Intermediate-Code Generation. Tehran. Tehran Polytechnic. Diakses pada 21 Mei 2026 dari https://www.m-zakeri.ir/Compilers/lectures/08_Intermediate-Code-Generation/.
- Lemone, Karen. *6.5.2 L-Attributed Attribute Grammars*. Worcester. Worcester Polytechnic Institute. Diakses pada 21 Mei 2026 dari <https://web.cs.wpi.edu/~cs544/PLT6.5.2.html>